

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
КАФЕДРА ІНФОРМАТИКИ ТА ПРОГРАМНОЇ ІНЖЕНЕРІЇ

Курсова робота з освітнього компоненту
«Моделювання систем. Курсова робота»
Тема: «Імітаційна модель системи студентського кафе на основі формального опису мережею масового обслуговування»

Керівник:
старший викладач
Стельмах Олександр Петрович

«Допущено до захисту»
_____

«___» _____ 2025 р.

Захищено з оцінкою
_____

Члени комісії:

_____

_____

Виконавець:
Бєліков Максим Романович
студент групи ІП-24
залікова книжка №КВ13878744
_____

«19» листопада 2025 р.

Інна СТЕЦЕНКО

Олександр СТЕЛЬМАХ

Київ – 2025

ЗАВДАННЯ

Виконати дослідження для такої системи:

Студентське кафе в університеті намагається поліпшити якість обслуговування. У час найбільшого напливу клієнтів, що спостерігається з 11.30 до 13.00, клієнти надходять групами по 1, 2, 3 та 4 студенти з ймовірностями 0,5; 0,3; 0,1 та 0,1 відповідно. Інтервали часу між прибуттям груп клієнтів мають експоненціальний закон розподілу з середнім значенням 30 секунд. Кожний клієнт обирає один із трьох маршрутів по кафетерію:

- пункт видачі гарячих блюд, пункт видачі напоїв, каса;
- пункт видачі холодних закусок, пункт видачі напоїв, каса;
- пункт видачі напоїв, каса.

Ймовірності вибору маршрутів складають 0,5; 0,4 та 0,1 відповідно. На видачі гарячих блюд та на видачі холодних закусок клієнтів обслуговують по одному. На видачі напоїв організовано самообслуговування так, що на цьому пункті ніколи не виникає черга. На пункті каса клієнтів обслуговують дві каси. У кожної каси своя черга клієнтів. З двох черг студенти обирають ту, що більш коротка. Перехід клієнтів з однієї черги в іншу не можливий.

Час обслуговування у пункті видачі гарячих блюд розподілений рівномірно в інтервалі від 50 до 120, у пункті видачі холодних закусок – рівномірно від 60 до 180, у пункті видачі напоїв – від 5 до 20. Час обслуговування у пункті каса розраховується в залежності від того на яких пунктах побував клієнт до каси. Якщо клієнт обслуговувався у пункті видачі гарячих блюд, то до часу його обслуговування на касі додається випадкова величина, що рівномірно розподілена в інтервалі від 20 до 40. Якщо клієнт обслуговувався у пункті видачі холодних блюд, то до часу його обслуговування на касі додається випадкова величина, що рівномірно розподілена в інтервалі від 5 до 15. Якщо клієнт обслуговувався у пункті видачі напоїв, то до часу його обслуговування на касі додається випадкова величина, що рівномірно розподілена в інтервалі від 5 до 10.

Визначити максимальний та середній час очікування в усіх чергах; максимальну та середню кількість клієнтів в усіх чергах; максимальний та середній час очікування клієнта у чергах окремо по кожному маршруту; максимальну кількість клієнтів в кафетерії; середню кількість клієнтів в кафетерії.

Для вищезазначеної системи:

1. розробити опис концептуальної моделі системи;
2. виконати формалізацію опису об'єкта моделювання в термінах визначеного у завданні формалізму;
3. розробити алгоритм імітації моделі дискретно-подійної системи відповідно до побудованого формального опису;
4. для доведення коректності побудованого алгоритму виконати верифікацію алгоритму імітації;
5. визначити статистичні оцінки заданих характеристик моделі, що є метою моделювання;
6. провести експериментальне дослідження моделі;
7. інтерпретувати результати моделювання та сформувати пропозиції щодо поліпшення функціонування системи;
8. зробити висновки щодо складності розробки моделі та алгоритму імітації на основі використаного формалізму, отриманих результатів моделювання та їх корисності.

АНОТАЦІЯ

Курсова робота на тему «Імітаційна модель системи студентського кафе на основі формального опису мережею масового обслуговування».

Текстова частина курсової роботи складається із вступу, 5 розділів, висновків, списку використаних джерел з 9 найменувань та 2 додатків, які викладено на 64 сторінках. У тексті роботи оформлено 22 рисунки, 6 таблиць.

У роботі поставлено завдання дослідити систему студентського кафе, розробити алгоритм імітації, провести верифікацію та експериментальне дослідження моделі, зробити висновки щодо отриманих результатів.

Імітаційна модель розроблена відповідно до формалізму мережі масового обслуговування з використанням об’єктно орієнтованого підходу мови програмування Python [7]. Була проведена верифікація моделі, результати свідчать про її коректну роботу.

Проведено дослідження вхідних параметрів мережі, за якими встановлено, що розроблена модель працює коректно.

Результати моделювання свідчать, що головними вузькими місцями системи є пункти видачі гарячих та холодних страв, де утворювались надмірні черги й значний час очікування клієнтів.

Зроблено висновки, що для усунення поточних проблем варто збільшити кількість каналів обслуговування до трьох на пункти видачі гарячих та холодних страв. Моделювання показало, що це дозволило усунути перевантаження, зменшити черги до прийнятного рівня та забезпечити стабільну роботу системи. Усі інші елементи системи витримували навантаження.

Ключові слова: імітаційна модель, алгоритм імітації, мережа масового обслуговування, кафе.

ЗМІСТ

ВСТУП 7

1 КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ 8

1.3 Загальний опис системи 8
1.4 Параметри моделі 9
1.5 Вхідні змінні 10
1.6 Вихідні змінні 11
1.7 Висновки розділу 13

2 ФОРМАЛІЗОВАНА МОДЕЛЬ СИСТЕМИ 14

2.1 Графічне представлення моделі 14
2.2 Початкові умови симуляції 15
2.3 Обрахунок вихідних змінних моделі 15
2.4 Висновки розділу 17

3 АЛГОРИТМІЗАЦІЯ ТА ПРОГРАМНА РЕАЛІЗАЦІЯ ІМІТАЦІЙНОЇ МОДЕЛІ СИСТЕМИ 18

3.1 Обґрунтування вибору мови програмування 18
3.2 Просування часу моделі 18
3.3 Реалізація черг 20
3.4 Сервери обслуговування 21
3.5 UML діаграма класів 22
3.6 Верифікація моделі 23
3.7 Висновки розділу 31

4 ЕКСПЕРИМЕНТАЛЬНЕ ДОСЛІДЖЕННЯ МОДЕЛІ 32

4.1 Аналіз поточної моделі 32
4.2 Експерименти з покращення моделі 35
4.3 Висновки розділу 42

5 ІНТЕРПРЕТАЦІЯ РЕЗУЛЬТАТІВ МОДЕЛЮВАННЯ, ФОРМУЛЮВАННЯ ВИСНОВКІВ ТА ПРОПОЗИЦІЙ 43

5.1 Інтерпретація результатів моделювання 43

5.2 Пропозиції покращення моделі 44

5.3 Висновки розділу 45

ВИСНОВКИ 46

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 48

ДОДАТКИ 49

Додаток А. Результати моделювань 49

Додаток Б. Лістинг програмного коду 54

ВСТУП

Сучасні підприємства сфери обслуговування дедалі частіше стикаються з необхідністю підвищення ефективності своєї роботи та задоволення потреб клієнтів. Особливо це стосується закладів громадського харчування, що функціонують у навчальних закладах, де попит на послуги має виражений піковий характер.

Однією з головних проблем таких систем є утворення черг у години найбільшого навантаження, що призводить до збільшення часу очікування, зниження задоволеності клієнтів і нераціонального використання ресурсів персоналу. Для розв’язання цих проблем варто дослідити процес обслуговування клієнтів та визначити можливі шляхи оптимізації. Одним із найефективніших методів для цього є імітаційне моделювання, яке дозволяє відтворити роботу системи, враховуючи випадковий характер надходження клієнтів, вибір маршрутів обслуговування та тривалості виконання операцій.

Метою курсової роботи є розробка та дослідження імітаційної моделі роботи студентського кафе університету з використанням дискретно-подійного підходу та формалізму мереж масового обслуговування.

У роботі буде використано мову програмування Python [7]. Візуалізація результатів здійснена за допомогою бібліотеки matplotlib [5], а графічний інтерфейс з використанням Tkinter [8]. Реалізація алгоритму відбуватиметься із застосуванням об’єктно-орієнтованого підходу, що сприятиме повторному використанню компонентів та розширюваності моделі.

Моделювання роботи студентського кафе дає можливість дослідити вплив різних факторів на швидкість обслуговування клієнтів, черги та загальну пропускну здатність системи. Отримані результати дозволяють оцінити ефективність роботи студентського кафе, виявити «вузькі місця» у системі обслуговування та розробити практичні рекомендації щодо покращення її функціонування.

1 КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ

Для представлення та деталізації процесу, що підлягає моделюванню, необхідно скласти концептуальну модель системи.

1.1 Мета моделювання

Метою моделювання є дослідження ефективності роботи студентського кафе університету у години найбільшого навантаження (11:30–13:00), а також аналіз наступних показників:

- максимальний та середній час очікування в усіх чергах (загалом);
- максимальна та середня кількість клієнтів в усіх чергах (загалом);
- максимальний та середній час очікування клієнта у чергах окремо по кожному маршруту;
- максимальна кількість клієнтів в кафетерії (одночасно присутніх);
- середня кількість клієнтів в кафетерії.

1.2 Складові елементи системи

При аналізі процесу обслуговування було сформовано наступний перелік елементів системи:

1. Клієнти – студенти, що прибувають до кафе.
2. Пункти обслуговування:
   - Видача гарячих страв – один канал обслуговування, утворюється черга нескінченної довжини.
   - Видача холодних закусок – один канал обслуговування, утворюється черга нескінченної довжини.
   - Видача напоїв – самообслуговування, черга не формується.
3. Дві каси – кожна має свій канал обслуговування та чергу нескінченної довжини. Клієнт обирає найкоротшу чергу та не може її змінювати.

1.3 Загальний опис системи

Об’єктом моделювання є студентське кафе, у якому процес обслуговування клієнтів складається з кількох послідовних етапів.

У години пік, що спостерігаються з 11:30 до 13:00, клієнти надходять до кафе групами розміром від однієї до чотирьох осіб. Вони обирають маршрут проходження через різні пункти обслуговування (видача гарячих страв, видача холодних закусок, видача напоїв). Далі кожен клієнт має бути обслуговований на одній з двох кас.

Усі події в системі відбуваються дискретно у часі й формують стохастичний процес, який описується засобами мережі масового обслуговування (ММО).

Схематичне представлення процесу обслуговування клієнта кафе наведено у BPMN [2] діаграмі (рисунок 1).

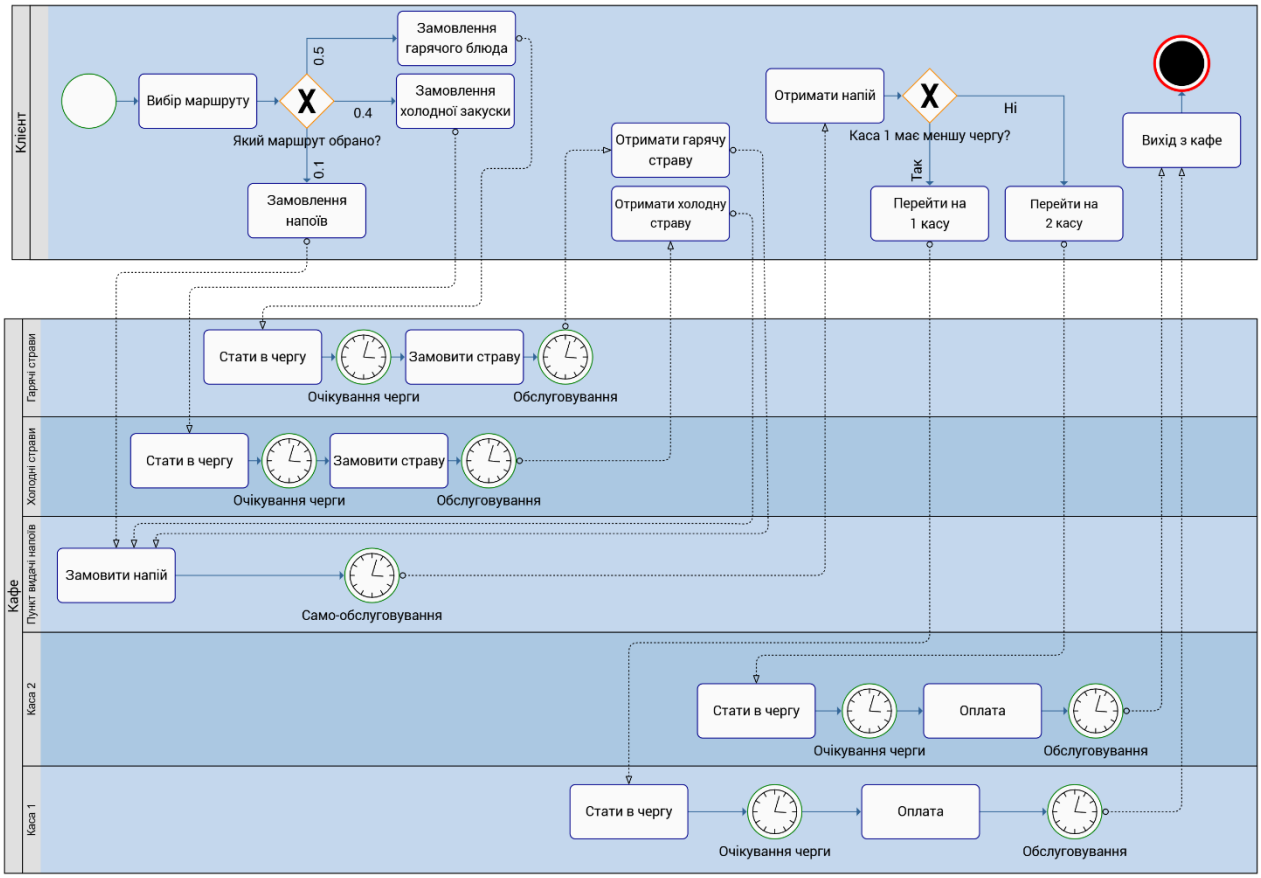


Рисунок 1.1. – BPMN діаграма процесу, що моделюється

1.4 Параметри моделі

На таблиці 1.1 зображено перелік параметрів мережі масового обслуговування, які не можуть бути змінені, але їх значення враховується в алгоритмі імітації.

Таблиця 1.1. – Параметри моделі

№	Назва параметра	Одиниця вимірювання	Діапазон	Закон розподілу
1	Розмір групи клієнтів	–	{1,2,3,4}	Дискретний, p = {0,5; 0,3; 0,1; 0,1}
2	Інтервал між прибуттями груп	секунди	(0; ∞)	Експоненційний, λ = 1/30
3	Маршрут клієнта	–	{1, 2, 3}	Дискретний, p = {0,5; 0,4; 0,1}
4	Кількість серверів пункту гарячих страв	–	1	–
5	Кількість серверів пункту холодних страв	–	1	–
6	Кількість серверів пункту видачі напоїв (самообслуговування)	–	∞	–

1.5 Вхідні змінні

Вхідні змінні будуть використані для верифікації моделі (таблиця 1.2).

Таблиця 1.2. – Вхідні змінні моделі

№	Назва змінної	Одиниця вимірювання	Діапазон	Закон розподілу
1	Час обслуговування на пункті видачі гарячих страв	секунди	[50; 120]	Рівномірний
2	Час обслуговування на пункті видачі холодних закусок	секунди	[60; 180]	Рівномірний
3	Час самообслуговування на пункті видачі напоїв	секунди	[5; 20]	Рівномірний
4	Додатковий час на касі після пункту гарячих страв	секунди	[20; 40]	Рівномірний
5	Додатковий час на касі після пункту холодних закусок	секунди	[5; 15]	Рівномірний
6	Додатковий час на касі після пункту видачі напоїв	секунди	[5; 10]	Рівномірний

1.6 Вихідні змінні

Відповідно до умови завдання було визначено перелік необхідних вихідних змінних для дослідження (таблиця 1.3). Для кожного наведеного значення обраховуються середнє, стандартне відхилення, максимальні та мінімальні значення за проведені ітерації симуляції.

Таблиця 1.3. – Вихідні змінні моделі

№	Змінна	Опис	Одиниця вимірювання
1	max_in_system	Максимальна кількість клієнтів одночасно у системі протягом симуляції	осіб
2	average_in_system	Середня кількість клієнтів у системі за час симуляції	осіб
3	max_total_queue_length	Максимальна загальна довжина всіх черг	осіб
4	time_average_total_queue_length	Середня загальна довжина всіх черг за час симуляції	осіб
5	global_avg_wait	Середній загальний час очікування клієнтів у системі	секунди
6	global_max_wait	Максимальний час очікування серед усіх клієнтів	секунди
7	route_avg_wait_A	Середній час очікування клієнтів маршруту A	секунди
8	route_max_wait_A	Максимальний час очікування клієнтів маршруту A	секунди
9	route_counts_A	Кількість клієнтів, що пройшли маршрут A	осіб
10	route_avg_wait_B	Середній час очікування клієнтів маршруту B	секунди
11	route_max_wait_B	Максимальний час очікування клієнтів маршруту B	секунди
12	route_counts_B	Кількість клієнтів, що пройшли маршрут B	осіб

Продовження таблиці 1.3

№	Змінна	Опис	Одиниця вимірювання
13	route_avg_wait_C	Середній час очікування клієнтів маршруту С	секунди
14	route_max_wait_C	Максимальний час очікування клієнтів маршруту С	секунди
15	route_counts_C	Кількість клієнтів, що пройшли маршрут С	осіб
18	queue_average_waits_hot_service	Середній час очікування клієнтів у черзі пункту видачі гарячих страв	секунди
19	queue_max_waits_hot_service	Максимальний час очікування клієнта у черзі пункту видачі гарячих страв	секунди
20	queue_max_lengths_cold_service	Максимальна довжина черги у пункті видачі холодних страв	осіб
21	queue_time_averages_cold_service	Середня довжина черги у пункті видачі холодних страв за час симуляції	осіб
22	queue_average_waits_cold_service	Середній час очікування клієнтів у черзі пункту видачі холодних страв	секунди
23	queue_max_waits_cold_service	Максимальний час очікування клієнта у черзі пункту видачі холодних страв	секунди
24	queue_max_lengths_cashier_1	Максимальна довжина черги до першого касира	осіб
25	queue_time_averages_cashier_1	Середня довжина черги до першого касира за час симуляції	осіб
26	queue_average_waits_cashier_1	Середній час очікування клієнтів у черзі до першого касира	секунди

Продовження таблиці 1.3

№	Змінна	Опис	Одиниця вимірювання
27	queue_max_waits_cashier_1	Максимальний час очікування клієнта у черзі до першого касира	секунди
28	queue_max_lengths_cashier_2	Максимальна довжина черги до другого касира	осіб
29	queue_time_averages_cashier_2	Середня довжина черги до другого касира за час симуляції	осіб
30	queue_average_waits_cashier_2	Середній час очікування клієнтів у черзі до другого касира	секунди
31	queue_max_waits_cashier_2	Максимальний час очікування клієнта у черзі до другого касира	секунди

Для зменшення кількості тексту та полегшення його сприйняття маршрути було названо літерами латинського алфавіту, де:

- Маршрут A: пункт видачі гарячих страв, пункт видачі напоїв, каса;
- Маршрут B: пункт видачі холодних страв, пункт видачі напоїв, каса;
- Маршрут C: пункт видачі напоїв, каса.

1.7 Висновки розділу

У даному розділі було розроблено концептуальну модель системи обслуговування клієнтів у студентському кафе. Було побудовано BMPN [2] діаграму, сформовано перелік параметрів моделі, вхідних та вихідних змінних. Це дозволило перейти до формалізації моделі системи.

2 ФОРМАЛІЗОВАНА МОДЕЛЬ СИСТЕМИ

У даному розділі представлено формалізовану модель системи, а також наведено формули для обчислення її вихідних змінних.

2.1 Графічне представлення моделі

Для обраного завдання був побудований формальний опис системи [1] студентського кафе мережею масового обслуговування (рисунок 1).

$$k = \begin{cases} 1, & x \le 0.5 \\ 2, & 0.5 < x \le 0.8 \\ 3, & 0.8 < x \le 0.9 \\ 4, & x > 0.9 \end{cases}, \text{ де } x = unif(0.5,\ 0.5)$$

$$m(q) = unif(7.5,2.5) + y, \text{ де } y = \begin{cases} unif(30,10), & q=1 \\ unif(10,5), & q=2 \\ 0, & q=3 \end{cases}$$

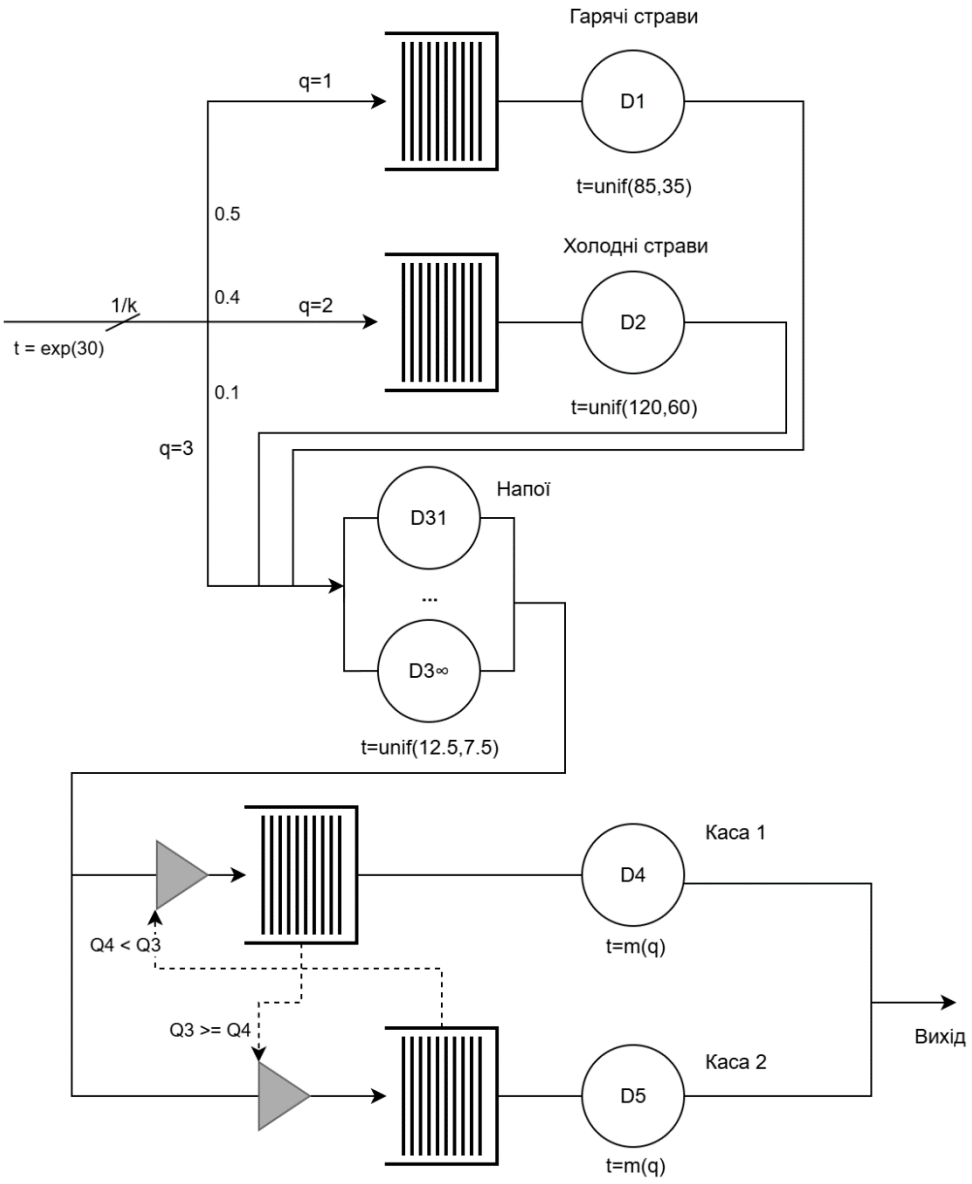


Рисунок 2.1. – Мережа масового обслуговування студентського кафе

Часові затримки надходження клієнтів у систему визначенні згідно з умовою завдання: експоненційний розподіл з середнім значенням 30 секунд. Через те що клієнти надходять групами від 1 до 4 осіб з ймовірностями, було використано дугу з числовим параметром $1/k$ (розгрупування), у якій вхідна змінна k обраховується за формулою відповідно до випадково згенерованого числа у межах від 0 до 1.

Кожен зі студентів обирає маршрут слідування мережею (q). Пункти видачі гарячих та холодних страв мають нескінченні черги, адже їх довжина не вказана вимогами завдання. Пункт видачі напоїв представлений як СМО з нескінченною кількістю каналів обслуговування, адже на ньому реалізовано самообслуговування.

Перед кожною з кас передбачені умови блокування, які залишають один маршрут завжди відкритим, тому пріоритети для них не задані. Час обслуговування залежить від обраного маршруту q та обраховується за формулою m(q).

2.2 Початкові умови симуляції

Прибуття першої групи клієнтів у систему відбувається у час t=0. Початково усі черги кафе порожні. Це припущення зроблено з міркувань того, що ми аналізуємо лише час пік клієнтів, де у кафе є проблеми з обслуговування та утворюються черги. До цього часу припускається, що кафе забезпечувало обслуговування усіх клієнтів.

2.3 Обрахунок вихідних змінних моделі

Для просування часу застосовувався принцип найближчої події тому, наведені формули для обрахунку вихідних значень враховуватимуть нерівні довжини інтервалів часу між якими збирається статистика.

Для усіх формул було використано наступні позначення:

T_{mod} – загальна тривалість моделювання;

t_i – моменти подій;

$\Delta t_i = t_{i+1} - t_i$ – інтервал між сусідніми подіями;

N_i – кількість клієнтів у системі протягом інтервалі $[t_i, t_{i+1})$;

$L_{\text{hot},i}, L_{\text{cold},i}, L_{\text{cash1},i}, L_{\text{cash2},i}$ — довжини черг протягом інтервалу $[t_i, t_{i+1})$;

$L_{tot,i} = L_{\text{hot},i} + L_{\text{cold},i} + L_{\text{cash1},i} + L_{\text{cash2},i}$ – довжина всіх черг інтервалу $[t_i, t_{i+1})$;

$\mathcal{S}$ – множина всіх клієнтів;

$\mathcal{S}_A, \mathcal{S}_B, \mathcal{S}_C$ – множини клієнтів, що пройшли маршрути A, B, C;

$\mathrm{w}_\mathrm{j}^\mathrm{A}, \mathrm{w}_\mathrm{j}^\mathrm{B}, \mathrm{w}_\mathrm{j}^\mathrm{C}$ – час очікування у чергах по маршрутах для кожного клієнта j;

$\mathrm{N}_{serv,A}, N_{serv,B}, N_{serv,C}$ – кількість обслугованих клієнтів маршрутів A, B, C.

Середня кількість клієнтів у системі за час симуляції:

$$N_{\text{avg}} = \frac{\sum_i N_i \Delta t_i}{T_{\text{mod}}} \tag{2.1}$$

Середня за часом загальна довжина всіх черг:

$$L_{\text{time_avg}} = \frac{\sum_i L_{tot,i} \Delta t_i}{T_{\text{mod}}} \tag{2.2}$$

Середня за часом загальна довжина кожної черги:

$$\begin{aligned} L_{\text{hot,avg}} &= \frac{\sum_i L_{\text{hot},i} \Delta t_i}{T_{\text{mod}}}, \\ L_{\text{cold,avg}} &= \frac{\sum_i L_{\text{cold},i} \Delta t_i}{T_{\text{mod}}}, \\ L_{\text{cash1,avg}} &= \frac{\sum_i L_{\text{cash1},i} \Delta t_i}{T_{\text{mod}}}, \\ L_{\text{cash2,avg}} &= \frac{\sum_i L_{\text{cash2},i} \Delta t_i}{T_{\text{mod}}}. \end{aligned} \tag{2.3}$$

Середній час очікування (усереднено по клієнтах):

$$W_{\text{avg}} = \frac{1}{N_{\text{serv}}} \sum_{j \in \mathcal{S}} w_j = \frac{\sum_i L_{tot,i} \Delta t_i}{N_{\text{serv}}} \tag{2.4}$$

Середній час очікування для окремих маршрутів:

$$W_{\text{avg},A} = \frac{1}{N_{\text{serv},A}} \sum_{j \in \mathcal{S}_A} w_j^A,$$
$$W_{avg,B} = \frac{1}{N_{\text{serv},B}} \sum_{j \in \mathcal{S}_B} w_j^B, \quad (2.5)$$
$$W_{avg,C} = \frac{1}{N_{\text{serv},C}} \sum_{j \in \mathcal{S}_C} w_j^C$$

Максимальні значення змінних обчислюються як максимум з масивів значень:

$$\mathrm{N}_{max} = \max_i N_i \quad (2.6)$$

$$L_{total_max} = \max_i L_{tot,i} \quad (2.7)$$

$$\mathrm{w}_{max_global} = \max_{j \in \mathcal{S}} w_j \quad (2.8)$$

За таким же принципом обраховано значення для кожної черги та маршруту окремо.

Кількість клієнтів, що пройшли відповідний маршрут, ідентична кількості обслугованих клієнтів цієї групи.

Усі наведені формули реалізовуються як дискретні суми під час подієвої симуляції: кожного разу при зміні стану системи фіксується вектор довжин черг і загальна кількість клієнтів N_i з врахуванням Δt_i; індивідуальні часи очікування збираються для кожного клієнта і використовуються у формулах середніх і максимальних значень.

2.4 Висновки розділу

У цьому розділі було представлено схему формалізованої моделі системи в рамках формалізму мережі масового обслуговування [1], а також наведено формули для обчислення вихідних змінних системи.

3 АЛГОРИТМІЗАЦІЯ ТА ПРОГРАМНА РЕАЛІЗАЦІЯ ІМІТАЦІЙНОЇ МОДЕЛІ СИСТЕМИ

У розділі наведено повний опис розробки алгоритму імітації, реалізації його основних компонентів, класів, ключових змінних. Обґрунтовано вибір мови програмування Python [6] як, розроблено та верифіковано імітаційну модель.

3.1 Обґрунтування вибору мови програмування

Для програмної реалізації імітаційної моделі була обрана мова програмування Python [7]. Ключовим фактором вибору стали доступні бібліотеки та простота їх інтеграції при розробці. За допомогою matplotlib [5] з наборів даних можна одразу генерувати зображення графіків для аналізу роботи системи. У поєднанні з бібліотекою TKinter [8] це дозволило зробити простий графічний інтерфейс з можливістю вказувати необхідні параметри симуляції та одразу виводити отримані вихідні показники та графіки роботи моделі. А бібліотека Pandas [6] дозволила зберігати результати моделювання у форматі csv без потреби розробки механізмів експорту даних власноруч.

Недоліком вибору є менша швидкість роботи програмного забезпечення та відсутність строгої типізації, порівняно з альтернативними мовами, такими як C++ [3], Java [4] тощо. Однак це не впливає на результати моделювання, тому доступні можливості для аналізу й експорту даних були пріоритетнішими.

3.2 Просування часу моделі

У програмній реалізації системи було застосовано принцип найближчої події для просування часу. Фактором вибору стала можливість точного просування часу, враховуючи лише моменти, коли в системі дійсно відбуваються зміни – надходження клієнтів, початок чи завершення обслуговування. Це дозволяє підвищити точність моделювання порівняно з альтернативними підходами, такими як фіксований крок часу.

Подіями системи є прибуття групи клієнтів, початок та завершення обслуговування кожним з пунктів обслуговування: гарячі страви, холодні страви, пункт видачі напоїв, дві каси. Після кожного просування часу збирається статистика для подальшого аналізу. За обробку відповідає клас SimulationEngine,

який містить методи для додавання нової події, реєстрації callback-функції для певного типу події, запуску та зупинки симуляції.

Псевдокод алгоритму просування часу

Вхідні змінні:

• until – момент часу, до якого виконується симуляція (необов'язковий параметр);

• max_events – максимальна кількість подій для обробки (необов'язковий).

Початок

running = True
paused = False
events_processed = 0
Поки running = True **та** event_queue не порожня:
Якщо paused = True:
Вийти з циклу
event = event_queue.get_next()
Якщо event = None:
Вийти з циклу
Якщо until задано **та** event.time > until:
Викликати statistic_handler(until)
Вийти з циклу
Якщо event.time > current_time:
Викликати _statistic_handler(event.time)
current_time = event.time
handler = event_handlers[event.event_type]
Якщо handler існує:
Викликати handler(event)
Збільшити events_processed на 1
Якщо max_events задано **та** events_processed >= max_events:
Вийти з циклу
running = False

Кінець

Основні програмні реалізації класів наведені в додатку Б.

Також було використано власний клас EventQueue, який реалізує структуру даних Heap, яка забезпечує додавання та отримання найближчої події з часовою складністю $O(\log n)$.

3.3 Реалізація черг

Одним із ключових компонентів імітаційної моделі є система черг, що описує процес очікування клієнтів перед обслуговуванням. Для забезпечення точного збору статистики та підтримки гнучкості при подальшому аналізі роботи системи було розроблено три класи: QueueStats, Queue та QueueMonitor.

При моделюванні було вирішено використовувати FIFO-черги (First-In, First-Out), що відповідає природному процесу обслуговування клієнтів у кафе. Для зберігання елементів застосовується структура deque, яка забезпечує додавання та вилучення елементів черги з часовою складністю O(1).

Збір статистики виконується окремим класом QueueStats, що дозволяє відокремити логіку управління чергою від обчислень показників. Для централізованого збору та агрегування статистики з усіх черг використовується клас QueueMonitor. Він здійснює знімок (snapshot) стану всіх черг у часі, а також надає методи для розрахунку усіх показників.

Загальний алгоритм роботи з чергами:

1. Додавання клієнта до черги (enqueue)
    - До структури deque додається новий об’єкт клієнта.
    - Фіксується момент його прибуття у словнику arrival_times.
    - Оновлюється статистика прибуттів.
2. Видалення клієнта з черги (dequeue)
    - З початку черги видаляється клієнт.
    - Розраховується час його очікування як різниця між поточним часом симуляції та моментом прибуття.
    - Значення записується у масив waiting_times.
    - Оновлюється середнє значення довжини черги та кількість обслугованих клієнтів.

3. Моніторинг черг (QueueMonitor)
    - Усі черги реєструються у моніторі за допомогою методу register_queue().
    - Метод take_snapshot() зберігає знімки довжин усіх черг у заданий момент часу.
    - На основі накопичених даних розраховуються узагальнені показники черг системи.

3.4 Сервери обслуговування

Ключовою складовою імітаційної моделі є сервери обслуговування, які відповідають за процес безпосереднього надання послуг клієнтам у різних пунктах системи. У контексті поточної системи кожен сервер відображає конкретний етап обслуговування – видачу гарячих страв, холодних страв, отримання напоїв чи оплату на касі.

Для забезпечення гнучкості моделі, усі сервери реалізовані на основі єдиного базового класу Server, від якого успадковуються конкретні реалізації:

- HotServicePoint – пункт видачі гарячих страв;
- ColdServicePoint – пункт видачі холодних страв;
- DrinkStation – пункт видачі напоїв (самообслуговування);
- Cashier – каса для розрахунку.

Алгоритм роботи базового класу сервера наступний:

1. У момент прибуття клієнта, він додається в чергу відповідного сервера. Якщо є вільний процес — обслуговування починається негайно.
2. Метод start_service призначає клієнта на вільний процес, зберігає час початку та генерує тривалість обслуговування.
3. Метод end_service звільняє процес та оновлює показники завантаженості.

Протягом симуляції кожним сервером відстежуються такі показники, як середній час обслуговування та коефіцієнт завантаженості кожного сервера.

Через те що час просування та збір статистики основане на подіях, то додавання клієнта в чергу, передача клієнта вільному процесу та звільнення процесу відбувається при настанні відповідної події. Наприклад, при початку

обслуговуванні сервером нового клієнта, додається подія в EventQueue зі згенерованим часом завершення обслуговування. При настанні цієї події спрацьовує доданий в SimulationEngine тригер на звільнення ресурсу.

Через специфіку завдання із блокування маршруту слідування необхідно було розробити додатковий клас CashierBank, який реалізує логіку вибору найкоротшої черги для клієнтів. У випадку, коли обидві каси мають однакову за довжиною чергу, обирається перша.

3.5 UML діаграма класів

Для наочного представлення структури програмного забезпечення було побудовано UML діаграму класів [9] (рисунок 3.1). Для перегляду повного коду можна скористатись посиланням на гітхаб на початку додатка Б.

У центрі системи знаходиться клас CafeteriaSimulation, який координує всі компоненти: сервери обслуговування, черги, генератор подій та збір статистики. Через взаємодію з класами SimulationEngine й EventQueue симуляція моделює потік клієнтів у часі, планує події та фіксує зміни станів системи.

Блок серверів обслуговування (HotServicePoint, ColdServicePoint, DrinkStation, Cashier, CashierBank) наслідує базовий клас Server, що визначає спільні методи для обробки клієнтів, обчислення часу обслуговування та використання ресурсів. Класи черг (Queue, QueueMonitor, QueueStats) відповідають за моделювання очікування клієнтів і збір даних щодо завантаженості.

Додаткові модулі, такі як StatsCollector, AggregatedStats, ExperimentRunner і SimulationVisualizer, забезпечують збір, аналіз і відображення статистичних результатів експериментів.

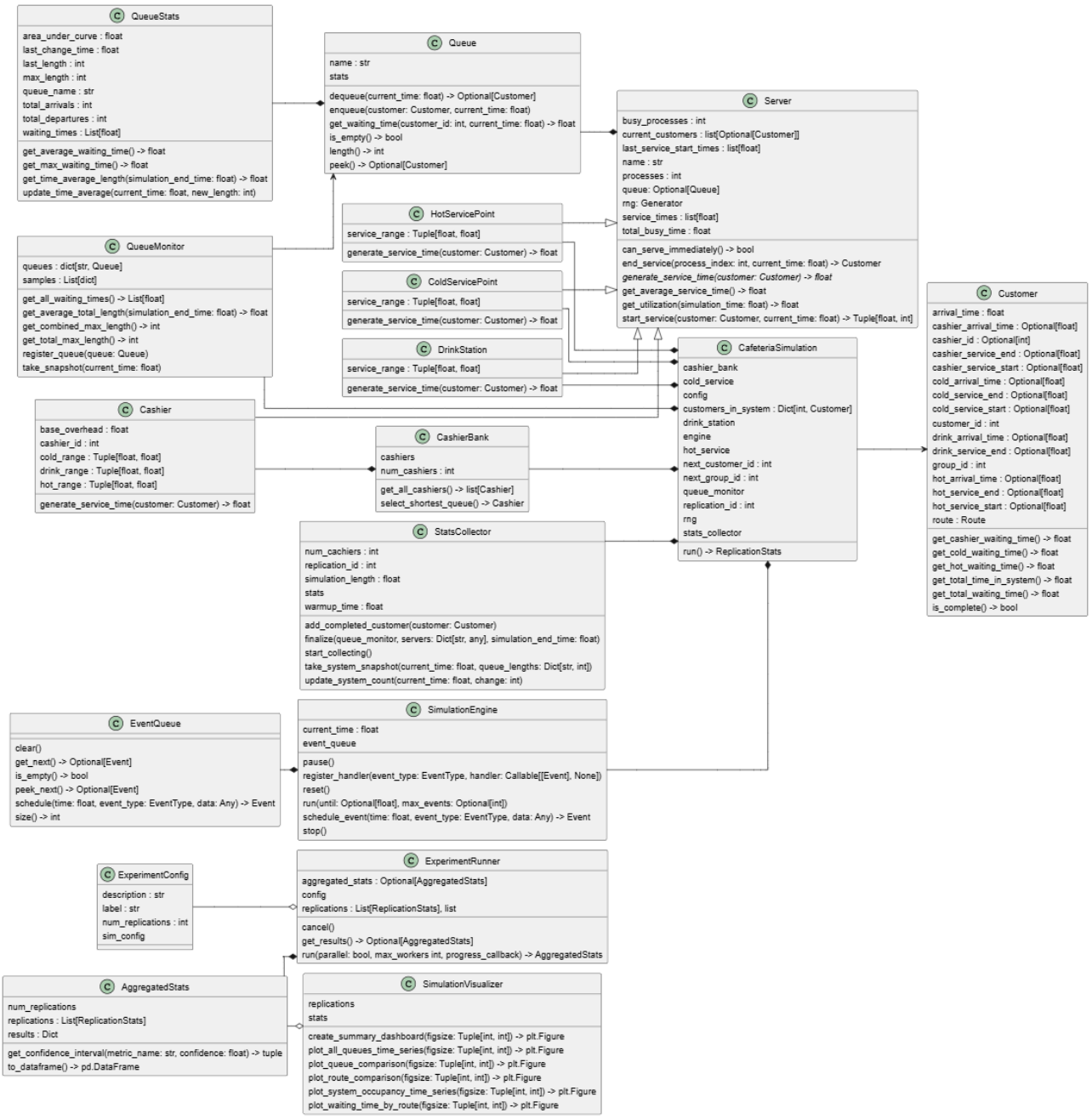


Рисунок 3.1. – UML діаграма класів

3.6 Верифікація моделі

Для перевірки коректності роботи мережі було проведено експерименти над кожним з вхідних з параметрів. Додатково було проведено тести на зміни кількості серверів обслуговування.

Кожен експеримент проводився на 100 запусках моделі та результуючі значення усереднювались. Кількість симуляцій для кожного експерименту була обрана для точності $\varepsilon = \sigma$ та довірчого інтервалу 99%. Це було обраховано з нерівності Чебишева [1]:

$$p = \frac{\sigma^2}{\varepsilon^2 (1-\beta)} \tag{3.1}$$

Для довірчого інтервалу 99% та точності $\varepsilon = \sigma$ кількість симуляцій дорівнює 100:

$$p = \frac{1}{1-0.99} = \frac{1}{0.01} = 100 \tag{3.2}$$

При побудові параметрів для експериментів було використано наступні правила: відносно великі значення були суттєво зменшені, а малі навпаки – збільшені. Через те що обрахунки вихідних параметрів кожного сервера обслуговування не відрізняються та кожен з них є базовим класом Server лише зі зміненою функцією розподілу, то достатньо було проводити по 1 експерименту для кожної змінної в більшу або меншу сторону. Можливі помилки обрахунків в одному з базових класів можуть легко бути виявлені на якомусь з експериментів.

Час симуляції для кожного експерименту однаковий і складає 5200 секунд (1.5 години). Він був взятий з умови завдання про те, що час найбільшого напливу клієнтів відбувається у проміжок з 11.30 до 13.00. Тобто моделюється інтервал у 1.5 години роботи системи.

Параметри кожного експерименту наведено у таблиці 3.1. Модель першого експерименту містила вхідні значення відповідно до умов завдання. Результати цього експерименту було вжито як еталонні (таблиці 3.2–3.3). Усі подальші інтерпретації і відносні зміни зроблено відносно значень першої базової моделі.

Таблиця 3.1 – Параметри проведених експериментів

Експеримент	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Час обсл. гарячі страви	[50;120]	**[10;20]**	[50;120]	[50;120]	[50;120]	[50;120]	[50;120]	[50;120]	[50;120]	[50;120]
Час обсл. холодні страви	[60;180]	[60;180]	**[10;20]**	[60;180]	[60;180]	[60;180]	[60;180]	[60;180]	[60;180]	[60;180]
Час обсл. напої	[5;20]	[5;20]	[5;20]	**[100;200]**	[5;20]	[5;20]	[5;20]	[5;20]	[5;20]	[5;20]
Час на касі після гарячих страв	[20;40]	[20;40]	[20;40]	[20;40]	**[100;200]**	[20;40]	[20;40]	[20;40]	[20;40]	[20;40]
Час на касі після холод. страв	[5;15]	[5;15]	[5;15]	[5;15]	[5;15]	**[100;200]**	[5;15]	[5;15]	[5;15]	[5;15]
Час на касі після видачі напоїв	[5;10]	[5;10]	[5;10]	[5;10]	[5;10]	[5;10]	**[100;200]**	[5;10]	[5;10]	[5;10]
Сервери на гарячих стравах	1	1	1	1	1	1	1	**5**	1	1
Сервери на холодних стравах	1	1	1	1	1	1	1	1	**5**	1
Середній час прибуття клієнтів	30	30	30	30	30	30	30	30	30	**10**

Вихідні значення кожного із запусків були усереднені за 100 симуляцій для кожного експерименту. Результати наведено у таблиці 3.2. Жовтим кольором підсвічений запуск моделі з параметрами без змін згідно з умовою завдання (перший експеримент). Червоним кольором підсвічено значення, які зменшились більше ніж на 10% порівняно з першим експериментом. Зеленим кольором відповідно підсвічені значення, які збільшились на 10% порівняно з 1 експериментом. Великі відносні зміни не можуть розглядатись як випадковість.

Таблиця 3.2 – Результати проведених експериментів

Вихідна змінна / Експеримент	1	2	3	4	5	6	7	8	9	10
max_in_system	185.61	92.97	102.93	190.46	194.46	192.84	261.63	92.81	107.49	767.71
average_in_system	94.67	46.83	51.48	97.39	99.11	99.24	131.65	47.67	54.95	385.65
max_total_queue_length	99.98	84.99	97.57	100.75	101.32	103.16	100.72	83.44	99.57	421.79
time_average_total_queue_length	91.75	43.55	48.72	91.02	94.94	95.22	127.41	42.40	49.78	382.48
global_avg_wait	1262.47	349.47	451.28	1218.62	1339.99	1293.20	1918.40	343.93	464.58	1198.99
global_max_wait	3455.38	3362.31	3125.35	3352.15	3528.24	3437.11	4210.19	3323.79	3138.47	4615.36
route_avg_wait_A	1590.38	42.29	1566.57	1539.72	1607.47	1650.35	2070.75	30.55	1584.87	2269.50
route_max_wait_A	3143.31	176.25	3125.35	3026.30	3148.93	3178.82	4075.84	154.90	3138.47	4527.74
route_counts_A	61.82	159.25	62.00	60.07	57.49	60.69	27.64	154.49	61.56	62.28
route_avg_wait_B	1712.02	1693.84	18.64	1643.93	1808.07	1684.01	2134.99	1686.29	22.39	2290.73
route_max_wait_B	3402.35	3362.31	81.87	3295.03	3464.49	3326.95	4114.18	3323.79	134.44	4574.83
route_counts_B	43.65	43.79	128.22	42.59	41.70	42.24	19.61	43.67	123.81	44.14
route_avg_wait_C	7.92	24.10	8.57	7.54	189.57	79.13	1320.03	21.05	8.63	7.29
route_max_wait_C	37.73	100.41	38.31	38.21	452.90	248.49	2586.09	84.37	38.05	42.29
route_counts_C	31.85	31.65	31.76	31.19	30.55	31.62	14.52	32.10	31.97	96.89
queue_max_lengths_hot_service	98.11	6.39	97.57	98.45	98.18	101.28	98.72	5.33	99.57	421.79
queue_time_averages_hot_service	49.26	0.49	48.18	48.51	48.01	50.43	48.70	0.24	49.14	210.84
queue_average_waits_hot_service	1630.05	15.97	1603.34	1620.47	1599.92	1635.97	1612.20	7.86	1621.26	2325.63
queue_max_waits_hot_service	3226.01	89.59	3073.74	3205.81	3202.33	3241.79	3196.63	86.61	3041.14	4544.21
queue_max_lengths_cold_service	84.56	84.99	4.80	85.48	85.53	85.81	84.65	83.44	5.37	342.35
queue_time_averages_cold_service	42.37	41.92	0.25	42.37	42.56	43.07	42.19	41.21	0.34	171.42
queue_average_waits_cold_service	1754.79	1728.18	10.33	1737.34	1751.67	1752.82	1749.00	1720.52	13.86	2349.38
queue_max_waits_cold_service	3424.49	3312.26	66.10	3326.21	3471.57	3401.20	3479.74	3294.56	121.23	4500.51
queue_max_lengths_cashier_1	1.06	4.12	1.80	1.12	5.77	3.59	37.90	3.40	1.81	1.47
queue_time_averages_cashier_1	0.13	0.74	0.26	0.14	2.41	1.05	18.51	0.63	0.26	0.21
queue_average_waits_cashier_1	5.35	26.47	8.18	6.32	184.41	75.61	1420.01	23.11	8.18	6.43
queue_max_waits_cashier_1	41.72	122.23	43.59	42.62	473.64	266.59	2825.28	101.09	44.40	43.47
queue_max_lengths_cashier_2	1.00	3.67	1.21	1.00	5.35	3.08	37.39	2.81	1.25	1.05
queue_time_averages_cashier_2	0.001	0.41	0.03	0.004	1.97	0.67	18.02	0.32	0.03	0.01
queue_average_waits_cashier_2	0.53	23.76	2.70	1.56	160.37	57.46	1408.47	20.11	3.07	1.26
queue_max_waits_cashier_2	4.49	107.56	29.22	14.86	438.37	238.60	2776.72	87.68	31.34	16.84

У таблиці 3.3 наведено зміну значень вихідних змінних у відсотках порівняно з першим експериментом.

Таблиця 3.3 – Відносна зміна вихідних змінних

Вихідна змінна / Експеримент	2	3	4	5	6	7	8	9	10
max_in_system	-49.91	-44.55	2.61	4.77	3.90	40.96	-50.00	-42.09	313.61
average_in_system	-50.53	-45.63	2.87	4.69	4.82	39.07	-49.64	-41.95	307.37
max_total_queue_length	-14.99	-2.41	0.77	1.34	3.18	0.74	-16.54	-0.41	321.87
time_average_total_queue_length	-52.53	-46.90	-0.80	3.48	3.78	38.86	-53.79	-45.75	316.85
global_avg_wait	-72.32	-64.25	-3.47	6.14	2.43	51.96	-72.76	-63.20	-5.03
global_max_wait	-2.69	-9.55	-2.99	2.11	-0.53	21.84	-3.81	-9.17	33.57
route_avg_wait_A	-97.34	-1.50	-3.19	1.07	3.77	30.21	-98.08	-0.35	42.70
route_max_wait_A	-94.39	-0.57	-3.72	0.18	1.13	29.67	-95.07	-0.15	44.04
route_counts_A	157.60	0.29	-2.83	-7.00	-1.83	-55.29	149.90	-0.42	0.74
route_avg_wait_B	-1.06	-98.91	-3.98	5.61	-1.64	24.71	-1.50	-98.69	33.80
route_max_wait_B	-1.18	-97.59	-3.15	1.83	-2.22	20.92	-2.31	-96.05	34.46
route_counts_B	0.32	193.75	-2.43	-4.47	-3.23	-55.07	0.05	183.64	1.12
route_avg_wait_C	204.45	8.21	-4.83	2294.33	899.47	16572.02	165.90	9.03	-7.88
route_max_wait_C	166.15	1.54	1.30	1100.53	558.69	6755.09	123.64	0.86	12.11
route_counts_C	-0.63	-0.28	-2.07	-4.08	-0.72	-54.41	0.78	0.38	204.21
queue_max_lengths_hot_service	-93.49	-0.55	0.35	0.07	3.23	0.62	-94.57	1.49	329.92
queue_time_averages_hot_service	-99.00	-2.19	-1.52	-2.54	2.38	-1.14	-99.52	-0.24	328.02
queue_average_waits_hot_service	-99.02	-1.64	-0.59	-1.85	0.36	-1.09	-99.52	-0.54	42.67
queue_max_waits_hot_service	-97.22	-4.72	-0.63	-0.73	0.49	-0.91	-97.32	-5.73	40.86
queue_max_lengths_cold_service	0.51	-94.32	1.09	1.15	1.48	0.11	-1.32	-93.65	304.86
queue_time_averages_cold_service	-1.06	-99.40	0.01	0.46	1.67	-0.42	-2.73	-99.19	304.63
queue_average_waits_cold_service	-1.52	-99.41	-0.99	-0.18	-0.11	-0.33	-1.95	-99.21	33.88
queue_max_waits_cold_service	-3.28	-98.07	-2.87	1.37	-0.68	1.61	-3.79	-96.46	31.42
queue_max_lengths_cashier_1	288.68	69.81	5.66	444.34	238.68	3475.47	220.75	70.75	38.68
queue_time_averages_cashier_1	470.40	104.48	11.46	1765.54	717.26	14251.71	392.26	101.97	61.19
queue_average_waits_cashier_1	394.35	52.77	18.00	3343.91	1311.95	26418.54	331.66	52.83	20.00
queue_max_waits_cashier_1	192.99	4.48	2.17	1035.37	539.06	6672.59	142.32	6.44	4.21
queue_max_lengths_cashier_2	267.00	21.00	0.00	435.00	208.00	3639.00	181.00	25.00	5.00
queue_time_averages_cashier_2	42626.18	2550.68	292.28	206924.65	69991.15	1891790.60	33784.57	2761.42	652.96
queue_average_waits_cashier_2	4351.17	405.06	191.38	29940.92	10662.80	263741.53	3666.92	475.54	135.77
queue_max_waits_cashier_2	2293.48	550.13	230.62	9654.96	5209.60	61690.27	1851.04	597.38	274.67

Результати першого експерименту свідчать про значне скупчення клієнтів в системі. Можна зробити теорію, що кількість клієнтів в системі зростала

лінійно, адже середня кількість клієнтів в системі (average_in_system) приблизно дорівнюють половині від максимально зафіксованої кількості клієнтів (max_in_system). Значення не є цілими числами, адже вони усереднені за 100 ітерацій. Також видно неймовірно великий максимальний час очікування клієнта 3455 секунд. За умовою час симуляції складає 5200 секунд. Тобто більшість інтервалу час пік клієнт стояв в чергах на обслуговування. Іншим цікавим фактом є те, що каса 2 майже ніколи не накопичувала черг. Відповідно до результатів максимальний розмір черги на цій касі складав 1. У середньому за час симуляцій довжина черги складала 0.001. Це означає що черга ймовірно виникала 1-2 рази на декілька секунд за весь час симуляції. Усе навантаження без проблем брала перша каса так, що на ній не утворювались черги довжиною більше за 1. І це наочно видно на показнику середньої довжини першої каси: 1.06. Більш детальний аналіз по показниках базової моделі буде зроблений у наступному розділі курсової роботи.

Другий експеримент зі зменшення часу обслуговування гарячих страв (з [50;120] до [10;20]) показав значущі зниження показників маршруту А: середній час очікування у черзі обслуговування гарячих страв скоротився на 99 %, середня кількість клієнтів в системі зменшилася приблизно на 45 %, загальний середній час очікування знизився більш ніж на 64 %. Одночасно зафіксовано відносне зростання часу очікування на касах: це перенесення вузького місця вперед по маршруту обслуговування: прискорення обробки на гарячій лінії призводить до більшої інтенсивності припливу на каси при незмінній пропускній здатності кас, що викликає локальне накопичення черг на касах. Через те що у першому досліді друга каса майже не була залучена (у середньому довжина черги складала 0.001), то тепер середня довжина черги 0.41. Перша каса здатна спокійно тримати майже все навантаження на собі.

Зменшення часу обслуговування холодних страв з [60;180] до [10;20] у 3 експерименті показав повну симетричну картину з 2 експериментом. Виявлені зміни відповідають очікуванням: усунення вузького місця на холодній лінії призводить до зниження локальних черг; при цьому частково підвищується

навантаження на каси. Однак через те, що цей маршрут менш популярний отримано трохи менші черги на касах, ніж у 2 експерименті.

Експеримент 4 зі збільшенням часу обслуговування напоїв з [5;20] до [100;200] показав значне збільшення індивідуального часу обслуговування напоїв, але вплив на глобальні метрики системи виявився неістотним. Це пояснюється тим, що на цьому пункті ніколи не може утворюватись черга, отже навіть суттєве збільшення його середнього часу не робить його системним вузьким місцем.

На експерименті 5 (збільшення часу на касі після гарячих страв з [20;40] до [100;200]) локальні показники каси зазнали різкого погіршення, що супроводжується помітним (але менш різким) зростанням глобального часу очікування. Це коректна реакція системи: локальне уповільнення на касі створює накопичення на вході в касовий вузол і, при достатньому обсязі потоку, передається на суміжні елементи мережі. Через те що каса 1 вже не витримує такого навантаження починає активно працювати 2 каса. На першому базовому експерименті 2 каса взагалі майже не працювала, тому спостерігаємо зростання довжини черги у 206924%, хоча фактично поганої ситуації нема, каси справляються успішно. Середня довжина черзі на 2 касі з приблизно 0.001 стала приблизно 1.97. Це і спричинило таке зростання відсотків.

Експеримент 6 (збільшення часу на касі після холодних страв з [5;15] до [100;200]) показав аналогічну поведінку до 5 експерименту: локальні черги та часи очікування зросли в рази, однак фактично обидві каси справляються успішно.

Експеримент 7 (збільшення часу на касі після видачі напоїв: з [5;10] [100;200]) показав ефект, коли каси вже не здатні справлятись з навантаженням. Середня довжина черги на 1 касі стала 18.51, а на піку досягала 37.9. Даний ефект пояснюється тим, що усі клієнти проходили через пункт видачі напоїв та його уповільнення призводить до масштабного зростання часу обслуговування та накопичення клієнтів. Неймовірні відсотки середньої довжини черги на 2 касі

1891790% пояснюються тим, що у першому експерименті вона взагалі майже не працювала (0.001), а тепер ледве справляється (18.02).

Експеримент 8 зі збільшенням кількості серверів на пункті видачі гарячих страв з 1 до 5 радикально зменшив черги та час очікування на 98 % у середньому на маршруті, середні та максимальні кількості клієнтів у системі знизились на 50 %, значне зниження глобального середнього часу очікування. Це очікувана кореляція: додаткові паралельні канали обслуговування підвищують пропускну здатність вузла, усуваючи його як вузьке місце.

Експеримент 9 (збільшення кількості серверів на холодній лінії з 1 до 5 проявив аналогічний до експерименту 8 ефект для холодної лінії: середній час очікування в черзі холодних страв знизився на 99%, середня кількість клієнтів в системі істотно знизилась. Поведінка повністю корелює з очікуваннями щодо паралелізації обслуговування.

Збільшення інтенсивності прибуття клієнтів за експоненційним розподілом з середнім інтервалом до 10 в експерименті 10 продемонструвала ще більше перевантаження системи: максимальна і середня кількість клієнтів в системі зросли понад 300%, середня довжина черг та середній час очікування значно збільшились. Це відповідає очікуваним результатам: при значному зростанні вхідної інтенсивності за незмінних сервісних ресурсів очікується перехід до стану з довгими чергами та значними затримками.

Отримані результати демонструють коректну роботу моделі як відображення мережі масового обслуговування з розподіленими сервісними вузлами. Зміни вхідних параметрів дають очікувані ефекти.

Зменшення середнього часу обслуговування або збільшення кількості паралельних серверів на конкретному маршруті призводить до різкого зменшення локальних черг і часу очікування; збільшення – до їх значного зростання. Локальні зміни можуть переносити вузьке місце на наступний етап обслуговування: при прискоренні одного вузла черги накопичуються на наступному, менш продуктивному вузлі.

Зміни інтенсивності прибуття мають мультиплікативний вплив на систему при незмінних сервісних ресурсах: збільшення потоку призводить до нелінійного наростання середньої кількості клієнтів в системі та середнього часу очікування.

Надані експерименти демонструють передбачувану, послідовну та теоретично обґрунтовану реакцію моделі на зміни параметрів. Таким чином, за умовами поставленого дослідження можна вважати, що модель працює коректно і придатна для подальшого використання при аналізі та оптимізації її архітектури.

3.7 Висновки розділу

Було обґрунтовано вибір Python [7] як мови програмування, наведено повний опис розробки алгоритму імітації, реалізації його основних компонентів, класів, ключових змінних. Побудовано UML діаграму класів [9] для відображення архітектури програмного забезпечення. Верифікація моделі показала, що вплив змінних коректно відповідає уявленню про процес функціонування системи. Модель працює справно.

4 ЕКСПЕРИМЕНТАЛЬНЕ ДОСЛІДЖЕННЯ МОДЕЛІ

Верифікації моделі показала, що вона придатна для подальшого використання при аналізі. Показники експериментів базової моделі показали, що необхідна оптимізація її архітектури. У цьому розділі було проведено експерименти та запропоновано можливості покращення обслуговування кафе.

4.1 Аналіз поточної моделі

Спершу було проаналізовано черги системи. На етапі верифікації моделі було спостережено явище неймовірно довгих черг пунктів видачі гарячих та холодних страв, що спричиняло високий час очікування. Графіки довжин черг з часом на рисунку 4.1. Блідим синім кольором відображено відхилення $\pm\sigma$. Через те що моделювання дискретно-подійних систем дає різні результати симуляції через фактор випадковості, було вирішено доречним показати інтервал, у якому будуть знаходитись результати з ймовірністю 99% (було проведено 100 симуляцій та усереднено результат).

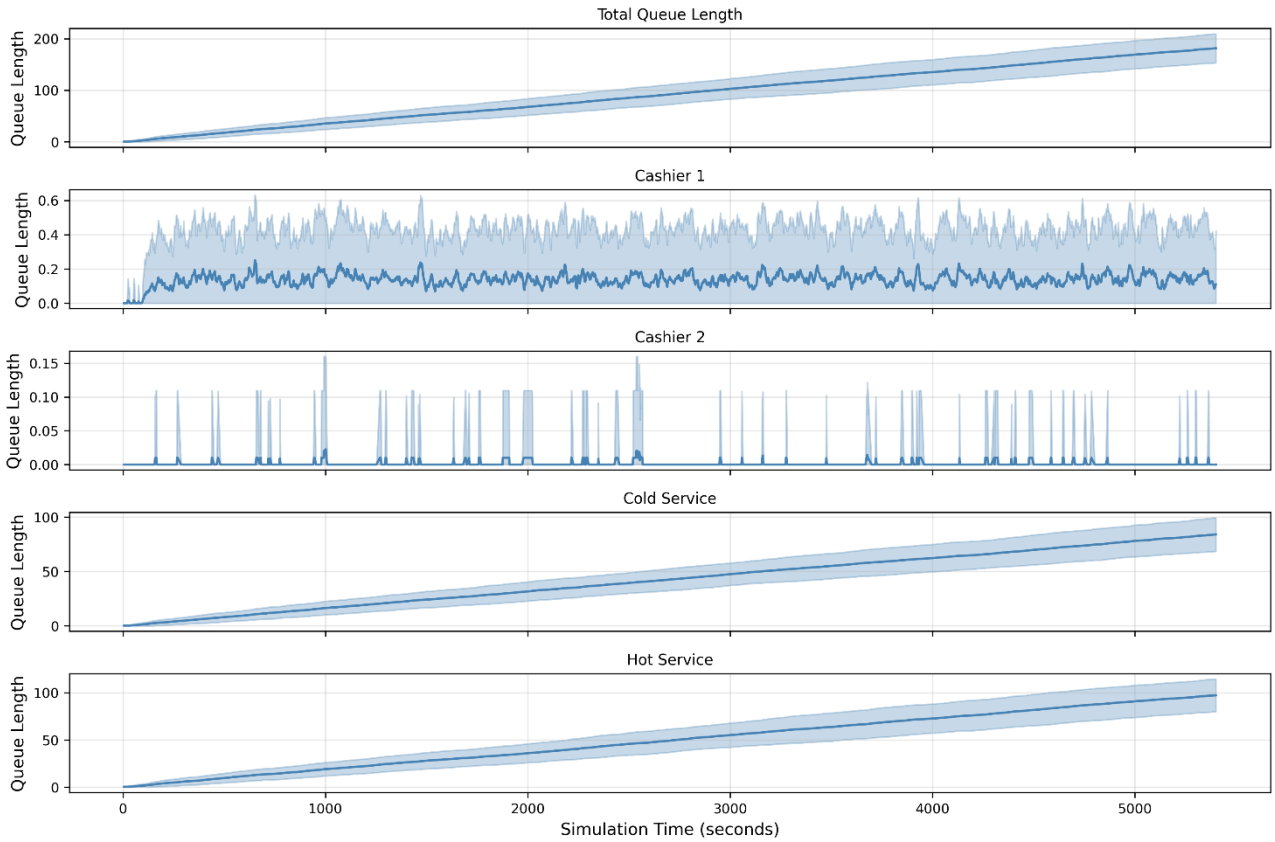


Рисунок 4.1 – Довжини усіх черг системи з часом

На рисунку 4.1 можна побачити, що на касі 1 майже не виникають черги. У середньому довжина черги в кожен момент часу знаходиться між 0.15-0.2 на 100 ітераціях. Якщо розглянути ситуацію з 2 касою, можна помітити невеличкі точки, де спостерігається черга. Блідо синій інтервал стандартного відхилення відсутній в місцях, де довжина черги дорівнює нулю. Так і має бути, адже в цей момент часу в жодній з симуляцій не виникала черга, а отже стандартне відхилення для масиву точок з нулів, буде 0. Бачимо, що каса 2 простоює.

Також видно, що довжини черг на пункті холодних та гарячих страв зростають з часом та не досягають сталих значень. Це ситуація, коли інтенсивність надходження в мережу масового обслуговування більша за інтенсивність обслуговування на цих пунктах, очевидно, спостерігається накопичення в чергах. Обчислене значення її середньої довжини буде залежати від часу моделювання, тому такий відгук моделі не є бажаним для дослідження.

Розглянемо середні та максимальні довжини черг, а також час очікування в них (рисунок 4.2).

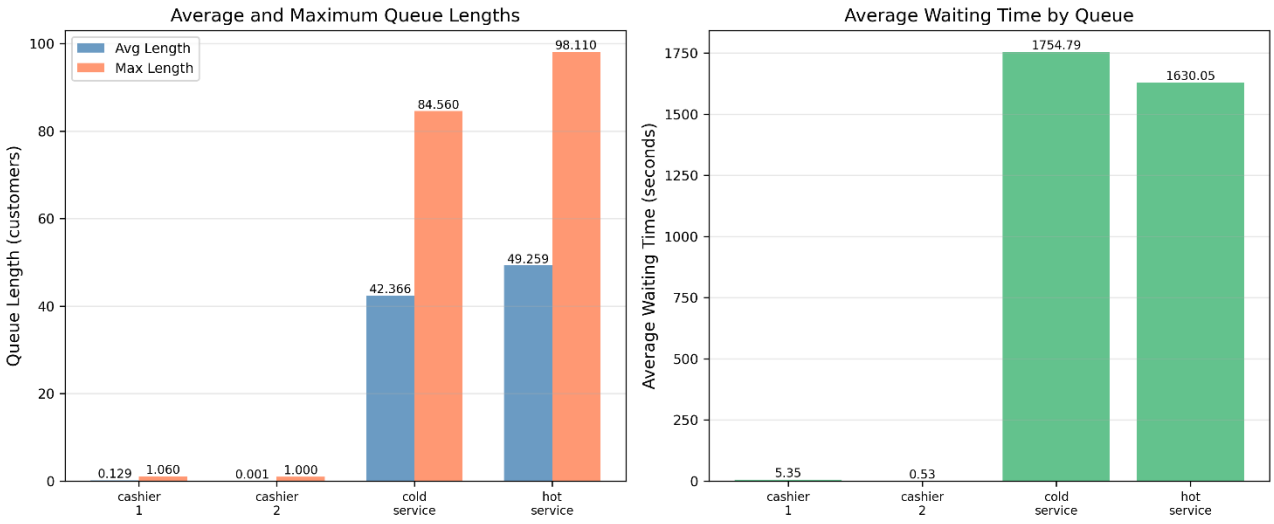


Рисунок 4.2 – Аналіз черг на кожному з пунктів обслуговування

Бачимо, що на касах не створюються черги, а навіть при їх виникненні час очікування в середньому складає 5.35 та 0.53 секунди на першій та другій касах відповідно. Спостерігаються неймовірно довгий час очікування на пунктах видачі страв.

Також важливо дослідити маршрути, за яким надходять клієнти (рисунок 4.3). Бачимо що усі клієнти маршруту С (пункт видачі напоїв, каса) завершують обслуговування менше ніж за 200 секунд. Час обслуговування на пунктах гарячих та холодних страв нагадує трапецієподібний розподіл (trapezoidal distribution).

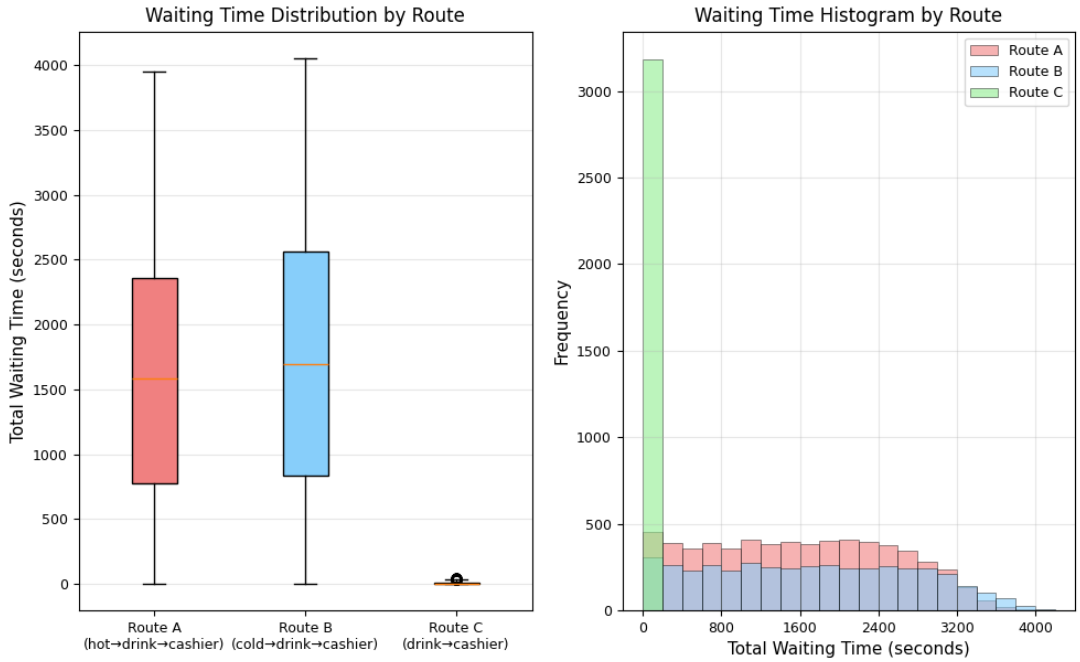
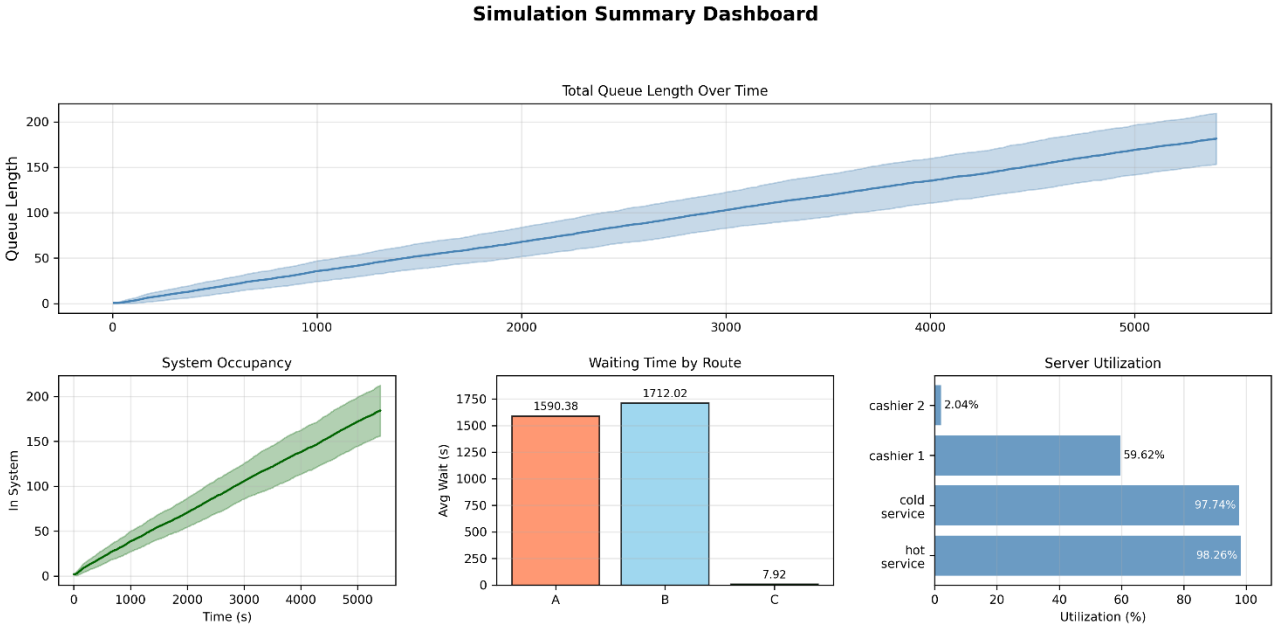


Рисунок 4.3 – Аналіз маршрутів обслуговування

Додатково було побудовано єдиний дашборд, на якому зображено усі основні метрики симуляції (рисунок 4.4). Ситуацію, що пункт видачі холодних та гарячих страв не здатен обробити такий вхідний потік клієнтів, додатково підтверджує навантаження на ці пункти. Вони працювали 97.74% та 98.26% часу відповідно.



Metric	Mean	Std Dev
Max in System	185.61	28.20
Avg in System	94.67	16.23
Max Total Queue	99.98	16.56
Avg Total Queue	91.75	16.22

Рисунок 4.4 – Дашборд з основними метриками симуляції

4.2 Експерименти з покращення моделі

Отже, поточна система не здатна функціонувати без значних накопичень черг, тому необхідно запропонувати варіанти її покращення.

У реальних умовах час обслуговування одним сервером на пунктах видачі страв не можливо суттєво зменшити через фізичну не спроможність працівників кафе працювати в рази швидше. Отже, наразі найкращим рішенням є збільшення кількості серверів обслуговування на цих пунктах.

Було збільшено кількість серверів обслуговування до 2 на пунктах видачі холодних та гарячих страв. Повна конфігурація запуску симуляції та її результати в додатку А. З них видно, що максимальна та середня кількість клієнтів у системі скоротилась вдвічі. Затримки по маршрутах значно зменшились.

Розглянемо графік довжин черг (рисунок 4.5). Зразу видно, що збільшення кількості серверів обслуговування на пунктах хоч і покращило систему, однак не призвело до досягнення сталого стану. Черги більше ніж 40 осіб на пунктах

гарячих та холодних страв неприпустимі для кафе. З іншого боку бачимо, що каса вже не простоює.

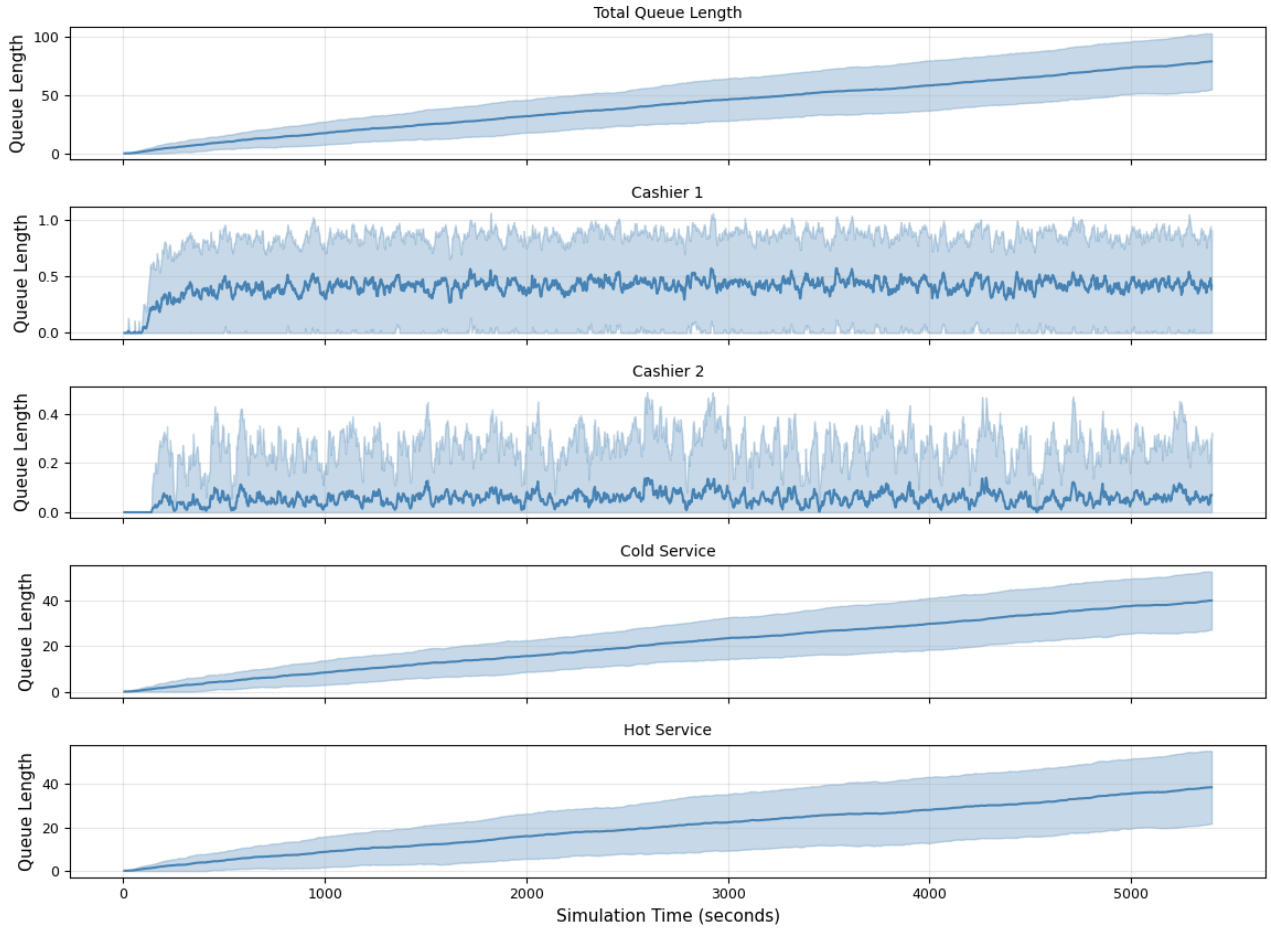


Рисунок 4.6 – Графік довжин черг покращеної системи

Отже, результати свідчать про потребу збільшення кількості серерів обслуговування на пунктах видачі холодних та гарячих страв. Було збільшено значення до 3. Детальні результати наведені у додатку А.

Розглянемо новий графік довжин черг (рисунок 4.7). На кінець симуляції середня довжина черги на пункті видачі гарячих та холодних страв досягла приблизно 5 осіб та 9 осіб відповідно. Ці черги є оптимальним результатом для системи кафе в годину пік, адже система здатна витримувати такі навантаження. Було досягнуто сталого стану для гарячих страв у приблизно 5 осіб черги. Видно легке зростання черги з часом для пункту холодних страв до 9. Однак через те, що час пік спостерігається виключно 1.5 години, то це є нормальним результатом її роботи. Пізніше буде перевірено систему на досягнення сталих значень.

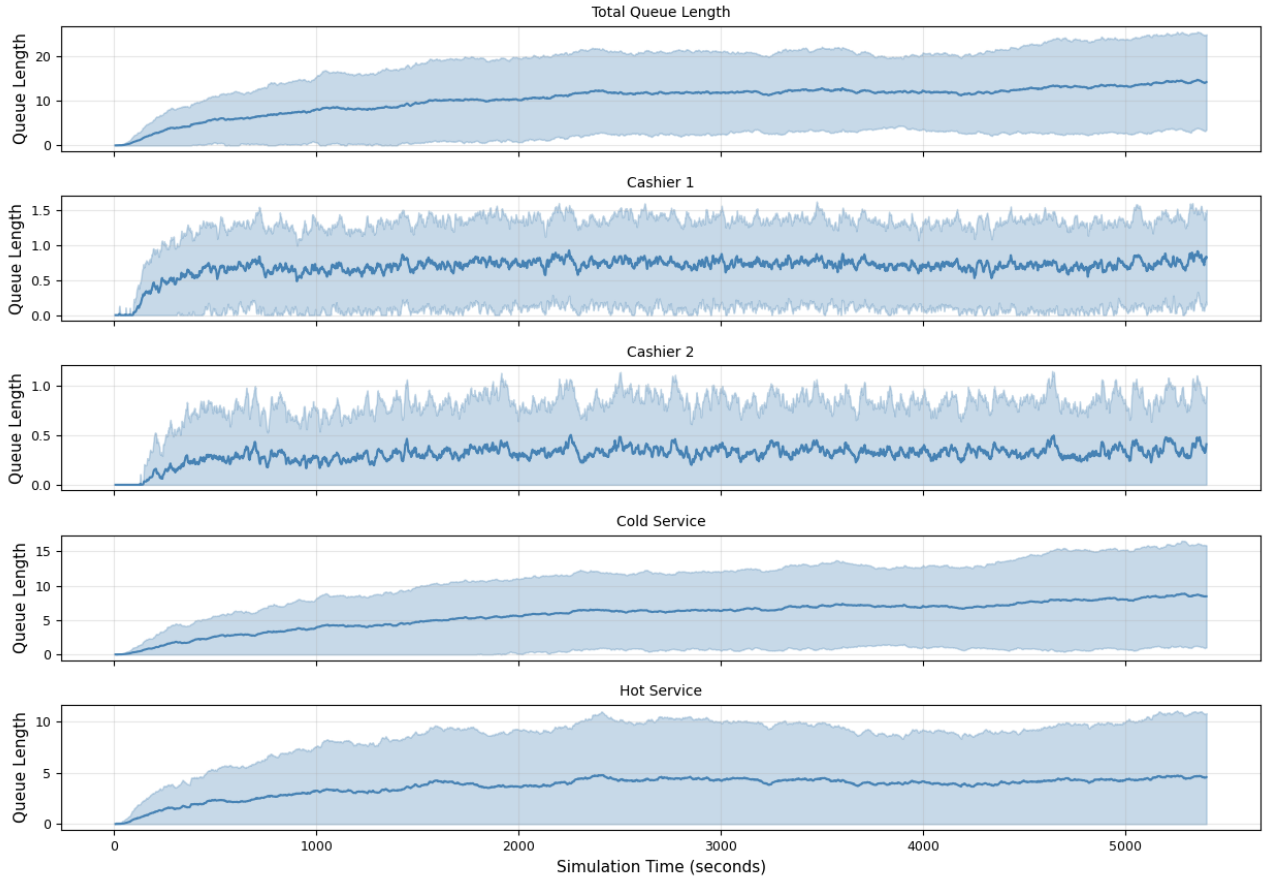


Рисунок 4.7 – Графік довжин черг вдруге покращеної системи

Розглянемо графік кількості клієнтів кафе (рисунок 4.8). Наочно видно логарифмічну тенденцію збільшення клієнтів з часом, що є цілком нормальним явищем. Стандартне відхилення є значним інтервалом, адже на кількість клієнтів в кафе впливають неймовірно багато факторів випадковості.

Якщо оцінити середні та максимальні довжини черг (рисунок 4.9), то видно, що найбільші черги досягались на пункті видачі холодних страв. Усереднене максимальне значення по 100 симуляцій склало 15.27 осіб. Не найкращий результат, але враховуючи факт, що ми розглядаємо лише пікове навантаження системи на проміжку 1.5 години це припустимо. Наймати додаткових працівників на обслуговування цього маршруту несе мало сенсу з погляду бізнесу, адже будуть значні простої у період звичайного напливу клієнтів протягом іншої частини доби.

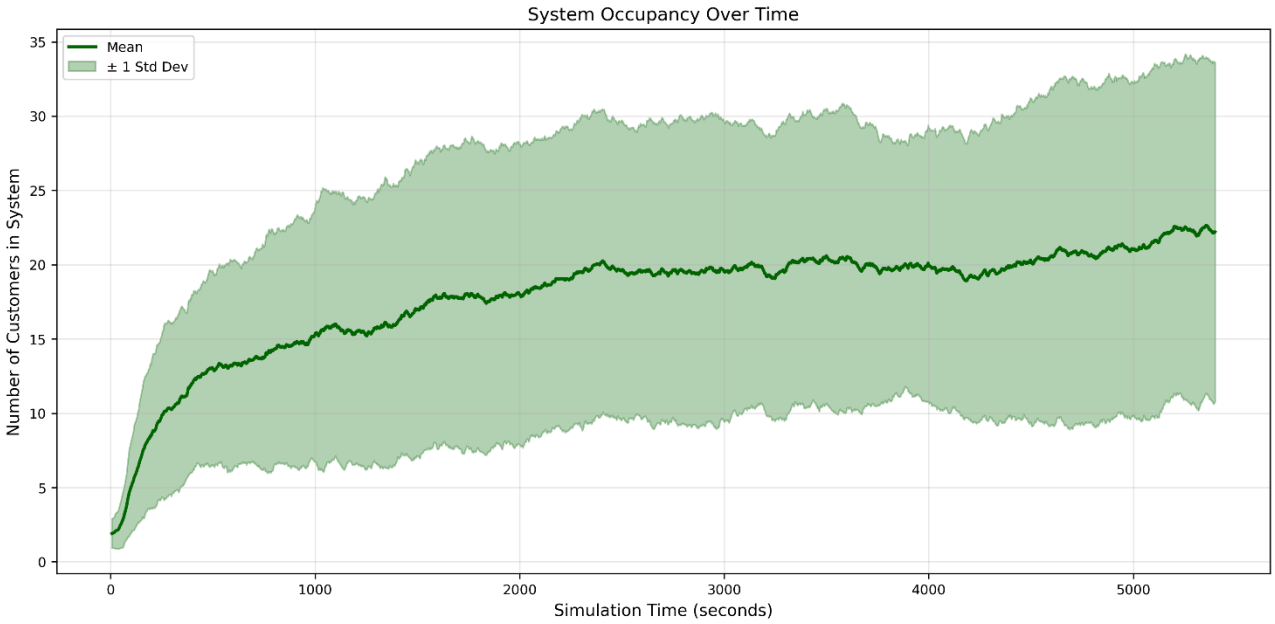


Рисунок 4.8 – Кількість клієнтів кафе з часом

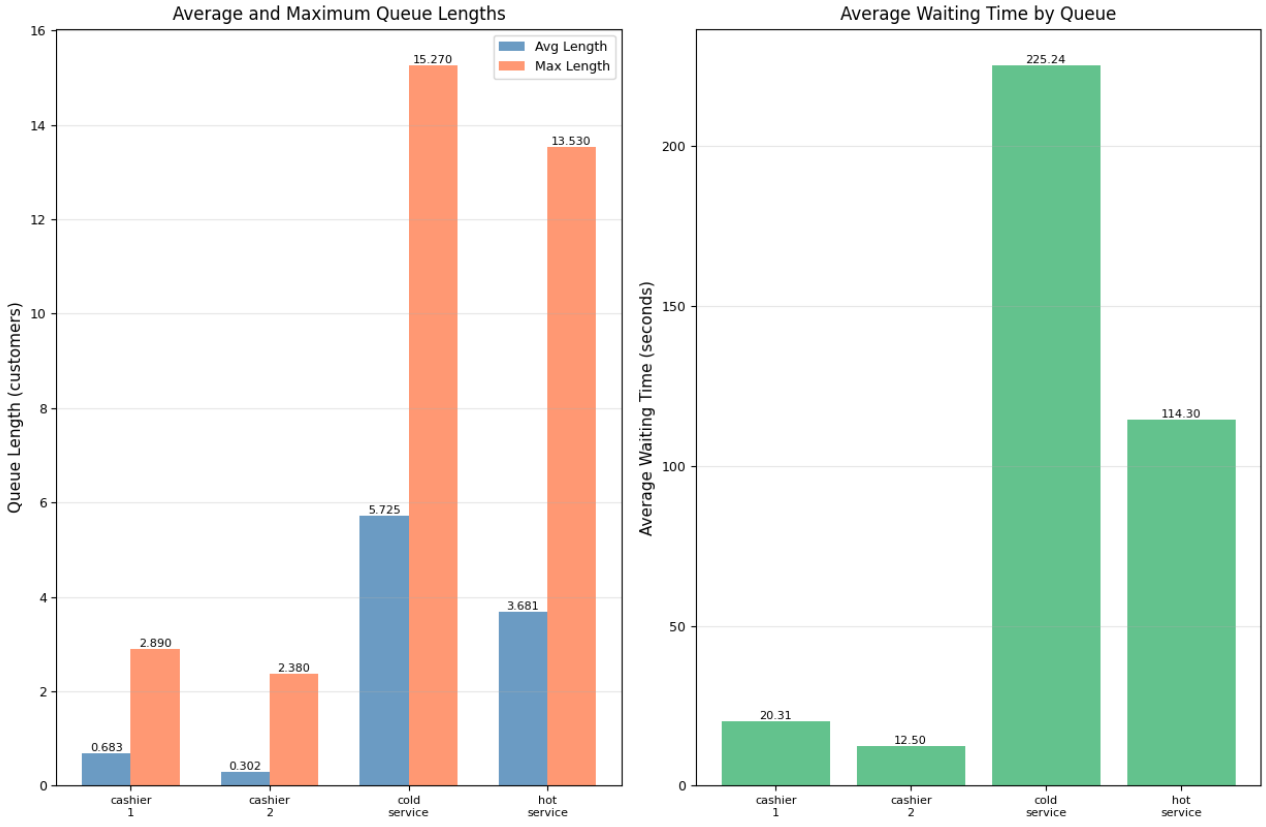


Рисунок 4.9 – Середня статистика по чергах

Розглянемо статистику за клієнтами що завершили обслуговування за маршрутами слідування (рисунок 4.10). Видно, що за усі 100 симуляцій найбільший час очікування був на сервісі холодних блюд і складав 1200 секунд

(20 хвилин), однак цей результат можна вважати випадковістю та рідким явищем. Зафарбована частина діаграми boxplot містить середні 50% усіх спостережень, нижній вус показує нижній квартиль 25% та верхній вус показує верхній квартиль. Деякі точки виходять за мережі типового діапазону (±1.5×IQR), адже істотно відрізняються від більшості. Ймовірно вони отримані у декількох малоймовірних ситуацій зі 100 симуляцій, де виникали значні черги.

Розгляд гістограм часу очікування показує схожість до експоненційного розподілу. Чітко видно найвищу щільність біля значень близьких до нуля та монотонне спадання. Маршрут С займає найменшу кількість часу. Більшість клієнтів пройшли весь маршрут менше ніж за 50 секунд. Маршрут з гарячими стравами показує швидшу тенденцію проходження ніж маршрут холодних страв.

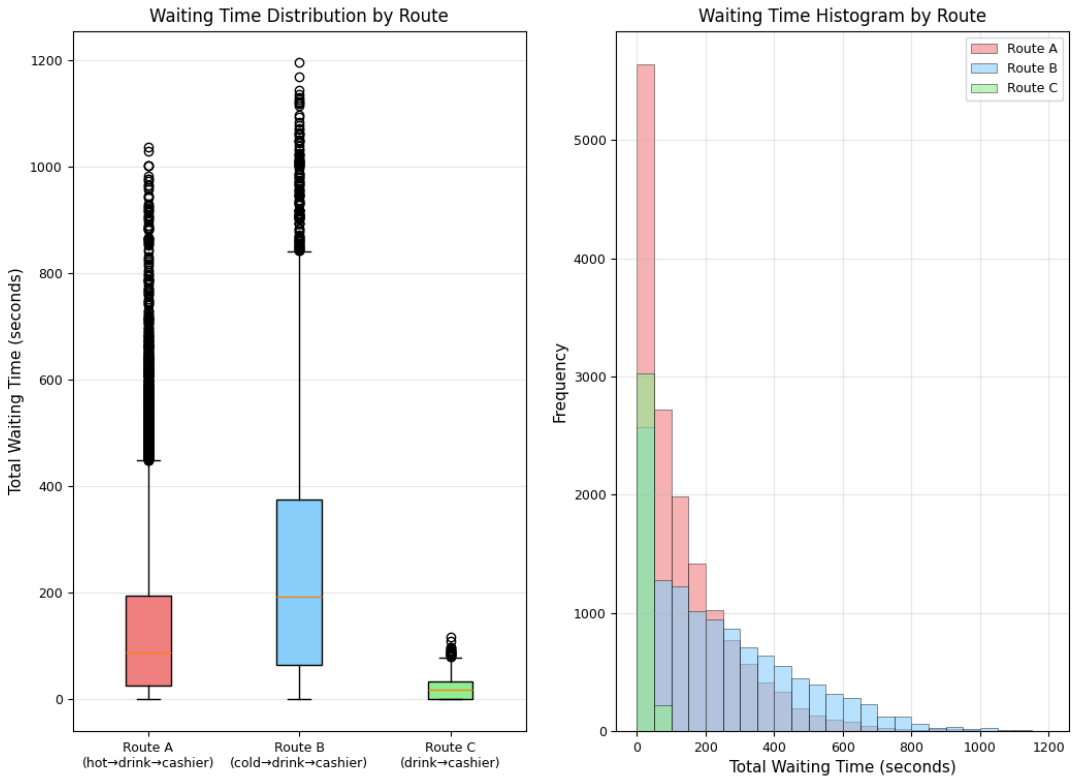


Рисунок 4.10 – Аналіз маршрутів слідування

Розглянемо середній час очікування по маршрутах відобразивши розмах стандартного відхилення (рисунок 4.11). Бачимо гарні результати. Найбільший час очікування на маршруті холодних страви, напої, каса, складав приблизно 240 секунд, що є дуже гарним показником. Це дуже гарний час повного обслуговування в годину пік.

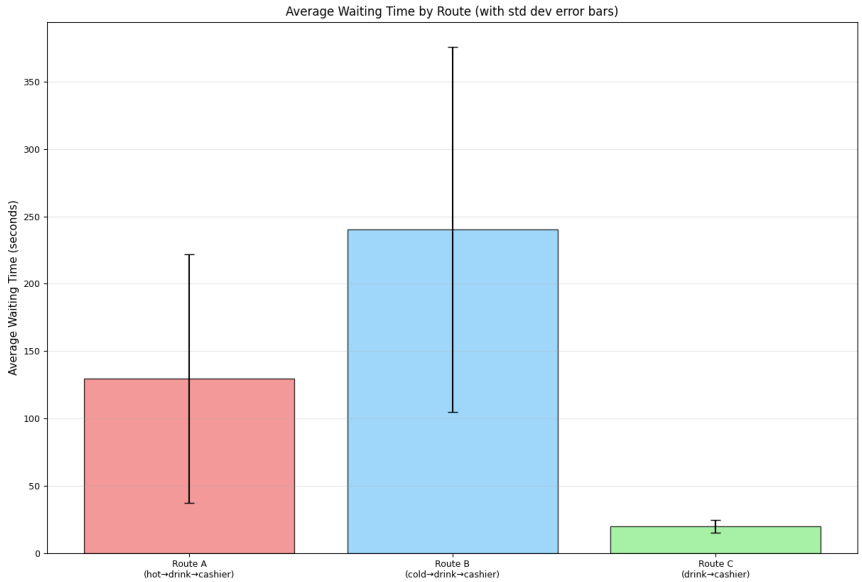
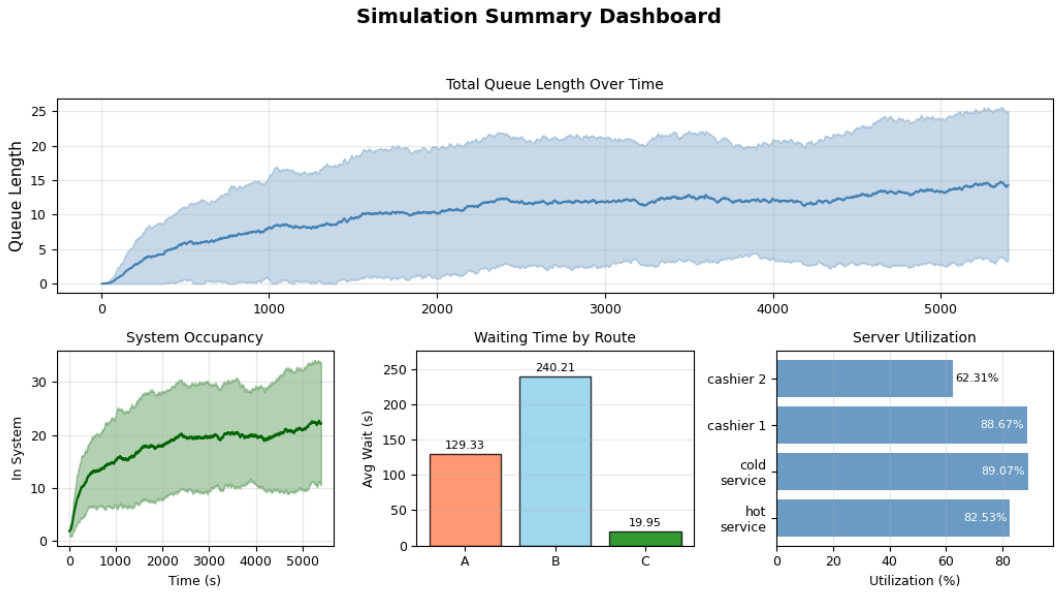


Рисунок 4.11 – Середній час очікування за кожним з маршрутів

Основна статистика цієї симуляції була відображена в загальному дашборді (рисунок 4.12). Бачимо, що сервери обслуговування майже постійно навантажені роботою. Каси повністю справляються з навантаженням, навіть мають запас по можливостях обслуговування. Детальні метрики покращеної моделі на рисунку 4.13.



Metric	Mean	Std Dev
Max in System	35.95	9.49
Avg in System	17.82	6.15
Max Total Queue	17.64	6.59
Avg Total Queue	10.39	5.77

Рисунок 4.12 – Загальна інформація про покращену модель

University Cafeteria Simulation

File Help

Configuration | Run Simulation | Results | Visualizations

Summary Statistics

Metric	Mean	Std Dev	Min	Max
max_in_system	35.95	9.49	21.00	71.00
average_in_system	17.82	6.15	9.88	47.91
max_total_queue_length	17.64	6.59	7.00	42.00
time_average_total_queue_length	10.39	5.77	3.13	39.35
global_avg_wait	162.07	77.98	52.46	519.92
global_max_wait	602.28	210.53	247.51	1195.79
route_avg_wait_A	129.33	92.25	33.44	615.76
route_max_wait_A	383.65	160.33	140.52	1036.66
route_counts_A	155.27	14.01	127.00	185.00
route_avg_wait_B	240.21	135.39	46.07	560.33
route_max_wait_B	567.92	224.51	188.64	1195.79
route_counts_B	119.29	7.78	99.00	137.00
route_avg_wait_C	19.95	4.84	9.91	40.19
route_max_wait_C	64.06	15.78	37.64	116.50
route_counts_C	32.55	6.92	19.00	50.00
queue_max_lengths_hot_service	13.53	5.95	5.00	37.00
queue_time_averages_hot_service	3.68	3.44	0.58	23.26
queue_average_waits_hot_service	114.30	90.84	22.82	595.76
queue_max_waits_hot_service	366.07	161.51	130.81	1013.36
queue_max_lengths_cold_service	15.27	6.53	5.00	42.00
queue_time_averages_cold_service	5.72	3.73	0.76	15.32
queue_average_waits_cold_service	225.24	135.12	36.04	536.88
queue_max_waits_cold_service	554.27	222.39	178.42	1165.60
queue_max_lengths_cashier_1	2.89	0.57	2.00	5.00
queue_time_averages_cashier_1	0.68	0.12	0.49	1.34
queue_average_waits_cashier_1	20.31	3.15	15.03	37.33
queue_max_waits_cashier_1	77.67	12.77	53.51	118.94
queue_max_lengths_cashier_2	2.38	0.56	2.00	4.00
queue_time_averages_cashier_2	0.30	0.10	0.17	0.92
queue_average_waits_cashier_2	12.50	2.92	9.00	29.55
queue_max_waits_cashier_2	63.22	14.27	40.99	109.59
server_utilization_hot_service	0.83	0.07	0.66	0.99
server_utilization_cold_service	0.89	0.06	0.71	0.98
server_utilization_drink_station	0.00	0.00	0.00	0.00
server_utilization_cashier_1	0.89	0.03	0.80	0.95
server_utilization_cashier_2	0.62	0.08	0.45	0.83

Рисунок 4.13 – Детальні метрики покращеної моделі

Було проаналізовано чи здатна покращена система досягати своїх сталих значень. Під час цього експерименту час моделювання склад 10800 секунд (3 години), однак перші 5400 секунд не враховувались у статистику.

Результати дожин черг такого експерименту дали відповідь, що черги не збільшують та не зростають з часом (рисунок 4.14). Видно, що загальна довжина черг у середньому трималась на одному рівні, як і інші черги. Черги на касах майже не виникали, адже середні значення менші за один. Довжина черги на пунктах гарячих та холодних страв складають приблизно 5 та 10 осіб відповідно.

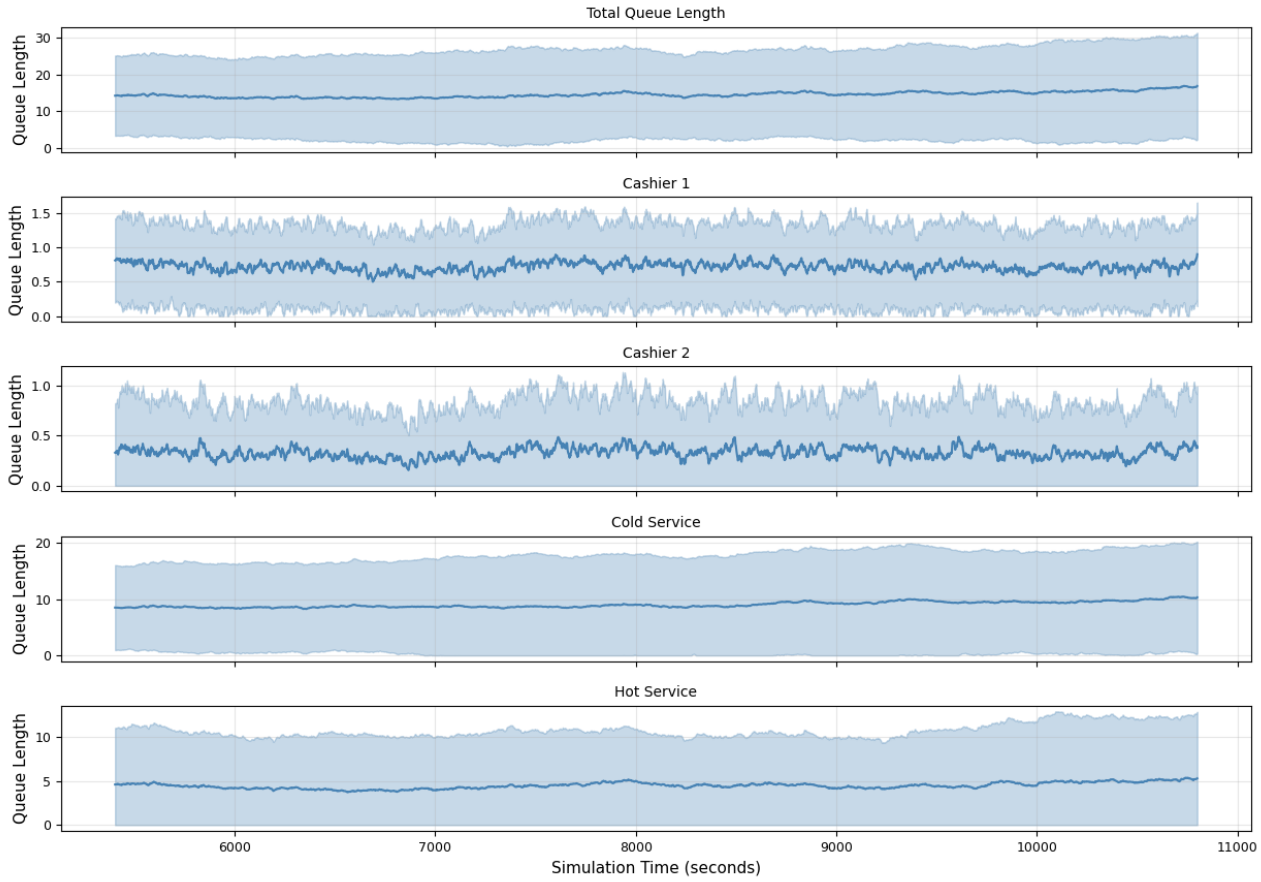


Рисунок 4.14 – Довжини черг під час перевірки досягнення сталих значень

4.3 Висновки розділу

Показники експериментів базової моделі підтвердили необхідність оптимізації її архітектури. У цьому розділі було проведено експерименти над декількома покращеними моделями та перевірено на досягнення сталих значень. Найкраща модель мала по 3 сервери на пунктах обслуговування гарячих та холодних страв. Було здійснено детальний її аналіз.

5 ІНТЕРПРЕТАЦІЯ РЕЗУЛЬТАТІВ МОДЕЛЮВАННЯ, ФОРМУЛЮВАННЯ ВИСНОВКІВ ТА ПРОПОЗИЦІЙ

Отримані результати продемонстрували необхідність покращення моделі у зв’язку нездатністю обробити потік клієнтів. Спочатку надано опис результатів моделювання, а потім запропоновано варіант покращення моделі.

5.1 Інтерпретація результатів моделювання

На основі аналізу експериментів (таблиці 3.2-3.3) та побудованого графіка 4.1 можна зробити висновок, що вузькими місцями системи є пункти видачі гарячих та холодних страв. Кількість черг на цих пунктах постійно зростала з часом. У середньому за 100 ітерацій моделювання максимальна кількість клієнтів у кафе приблизно була 185 осіб. Найбільшою була черга пункті видачі гарячих страв і за ітерації моделювання у середньому її максимальне значення досягало 98 осіб. Для пункту холодних страв довжина сягала 84 особи. При цьому каси не мали значного навантаження, черги понад 2 осіб не утворювались, а друга каса майже весь час не мала клієнтів. Проблем на пункті видачі напоїв теж не було виявлено.

Результати свідчать про те, що більшість клієнтів застрягло у системі перед пункти видачі гарячих та холодних страв. Найпопулярніший маршрут (гарячі страви, напої, каса) успішно завершувало у середньому 61 особа, однак черга перед пунктом видачі гарячих страв сягала близько 98 осіб. Тобто за 1.5 години пікового навантаження кафе, опрацьованих клієнтів на маршруті значно менше, ніж загально надійшовших. Максимальний час очікування клієнта сягав аж 3455 секунд (57 хвилин). Враховуючи, що моделювались лише 1.5 години, то у після час піку клієнти що лишились в чергах очікували б ще більше часу.

Пропускної здатності кас вистачає для клієнтів, що на них надходять. Середній час очікування в чергах складає 5 секунд, адже в більшості випадків вони навіть не утворюються.

Отже, система є недієздатною через перевантаження на пунктах видачі гарячих та холодних страв. З погляду бізнесу, кафе втрачає свою репутацію та

отримає невдоволені відгуки клієнтів, які чекають понад годину в чергах. Відповідно це вплине на загальний прибуток та популярність.

5.2 Пропозиції покращення моделі

Було проаналізовано декілька можливих варіантів покращення системи. Через те що час обслуговування одним сервером на пунктах видачі страв не можливо суттєво зменшити через фізичну не спроможність працівників кафе працювати в рази швидше, найкращим рішенням є збільшення кількості працівників на цих проблемних пунктах.

Було досліджено можливість паралельної обробки двох клієнтів на пунктах видачі холодних та гарячих страв. Це зменшило черги майже вдвічі. Кількість клієнтів в системі зменшилась зі 185 до 88 осіб. Однак черги все одно постійно зростали та не досягали сталих значень. Каси не відчули суттєвого навантаження. Черги стали виникати частіше, але вони є незначними, до 2 осіб. Час очікування виріс до 13 секунд на першій касі та до 4 секунд на другій касі.

Далі було розглянуто варіант з 3 каналами обслуговування на пунктах видачі холодних та гарячих страв. Це дозволило позбутись вузького місця на цих пунктах. Максимальна кількість клієнтів у кафе у середньому була 36 осіб. Середні довжини черг були 3.68, 5.72, 0.68 та 0.3 для пунктів видачі гарячих страв, холодних страв, каси 1 та каси 2. Каси легко стримують наплив клієнтів. Однак, якщо прибрати одну з них, це спричинить вузьке місце. У середньому клієнти стояли в чергах на маршрутах обслуговування 61.82, 43.65 та 7.92 секунд. Кількість клієнтів, що завершили обслуговування на момент закінчення симуляції у середньому зросла зі 61.82 до 155.27 на першому маршруті, 43.65 до 119.29 на другому маршруті, та з 31.85 до 32.55 на третьому маршруті.

Було зроблено симуляцію роботи покращеної моделі для досягнення сталих значень. Тобто, якщо кафе буде працювати при такому навантаженні увесь час, то черги на касах майже не виникатимуть (середні значення менші за один), довжина черг на пунктах гарячих та холодних страв складатимуть приблизно 5 та 10 осіб відповідно. Система є оптимальною для поточного навантаження.

5.3 Висновки розділу

У результаті моделювання встановлено, що головними вузькими місцями системи є пункти видачі гарячих та холодних страв, де утворювались надмірні черги й значний час очікування клієнтів. Збільшення кількості каналів обслуговування до трьох на кожному з цих пунктів дозволило усунути перевантаження, зменшити черги до прийнятного рівня та забезпечити стабільну роботу системи.

ВИСНОВКИ

Під час виконання курсової роботи було досліджено університетське кафе. Метою роботи було проаналізувати ефективність його роботи та запропонувати покращення системи.

Для виконання поставленого завдання у першому розділі було досліджено предметну область, а саме: виявлено параметри системи, вхідні та вихідні змінні, розроблено концептуальну модель системи обслуговування клієнтів та побудовано BMPN [2] діаграму. Це дозволило перейти до формалізації моделі системи.

У другому розділі було представлено схему системи в рамках формалізму мережі масового обслуговування [1], а також наведено формули для обчислення кожної з вихідних змінних.

У третьому розділі було обґрунтовано вибір Python [7] як мови програмування, наведено повний опис розробки алгоритму імітації, реалізації його основних компонентів, класів, ключових змінних. Побудовано UML діаграму класів [9] для відображення архітектури програмного забезпечення. Верифікація моделі показала, що зміна вхідних параметрів коректно відображається на вихідних та отримані значення відповідають уявленню про процес функціонування системи. Модель працює справно. Виявлено проблеми поточної системи.

У четвертому розділі детально досліджено кожну вихідну змінну. Головними вузькими місцями системи є пункти видачі гарячих та холодних страв, де утворювались надмірні черги й значний час очікування клієнтів. Показники експериментів базової моделі підтвердили необхідність оптимізації її архітектури. Також проведено експерименти над декількома покращеними моделями та перевірено на досягнення сталих значень. Збільшення кількості каналів обслуговування до трьох на кожному з цих пунктів дозволило усунути перевантаження, зменшити черги до прийнятного рівня та забезпечити стабільну роботу системи.

У п’ятому розділі надано висновки результатів моделювання та коротко обґрунтовано пропозицію про покращення моделі на основі попередніх досліджень.

Поставлена мета курсової роботи виконана.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Стеценко І. В., Дифучина О. Ю., Дифучин А. Ю. Рекомендації до виконання курсової роботи : навч. посіб. Київ : КПІ ім. Ігоря Сікорського, 2024. 109 с.
2. BPMN specification. Business Process Model and Notation. URL: https://www.bpmn.org (дата звернення: 02.11.2025).
3. C++ programming language. URL: https://cplusplus.com (дата звернення: 02.11.2025).
4. Java. Oracle. URL: https://www.java.com/en (дата звернення: 02.11.2025).
5. Matplotlib. Visualization with Python. URL: https://matplotlib.org (дата звернення: 02.11.2025).
6. Pandas. Python Data Analysis Library. URL: https://pandas.pydata.org (дата звернення: 02.11.2025).
7. Python. Python.org. URL: https://www.python.org (дата звернення: 02.11.2025).
8. TkInter. TkDocs. URL: https://tkdocs.com (дата звернення: 02.11.2025).
9. UML. Unified Modeling Language. URL: https://www.uml.org (дата звернення: 02.11.2025)..

ДОДАТКИ

Додаток А. Результати моделювань

Усі результати моделювань, експерименти для верифікації, графіки, значення вихідних змінних та процес моделювання завантажено на Google Drive: https://drive.google.com/drive/folders/1m61N8D3y-GmIX0FR39RQgpmwc2SEohFU

Було вивантажено на Google Drive всі графіки, вихідні та вхідні метрики кожного експерименту, порівняльні Excel таблиці, які використовувались у тексті пояснювальної записки.

На рисунку 1 зображено конфігурацію базової моделі.

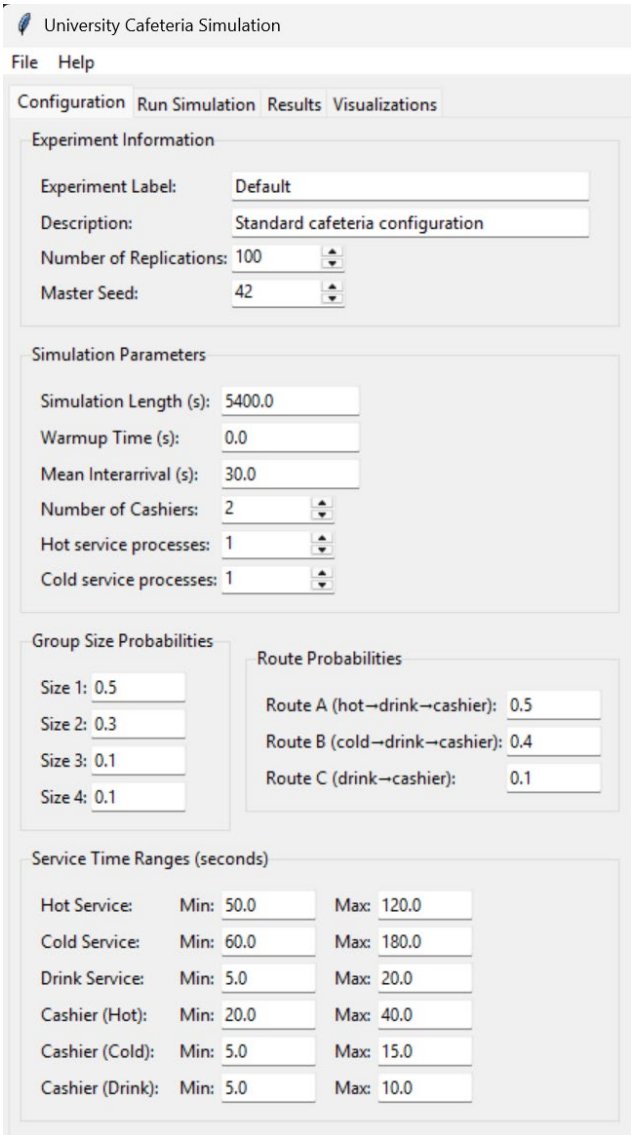


Рисунок 1 – Конфігурація базової моделі

На рисунку 2 зображено вихідні результати моделювання у вигляді CSV файлу.

	A	B	C	D	E
1	**Metric**	**Mean**	**Std**	**Min**	**Max**
2	max_in_system	185.61	28.198751673781523	113.0	255.0
3	average_in_system	94.66848458370714	16.22973667639801	60.61537089585936	139.8006506879483
4	max_total_queue_length	99.98	16.564523366020357	61.0	141.0
5	time_average_total_queue_length	91.75477817807372	16.21847482021264	57.70068838913971	136.86552565196737
6	global_avg_wait	1262.472896641131	129.82542882630543	944.8231091992336	1588.5293970142227
7	global_max_wait	3455.3764936402895	321.4579685294319	2741.0892070036916	4054.4655481342425
8	route_avg_wait_A	1590.3771343138317	198.0206162994231	1069.115430724953	2010.3843709113535
9	route_max_wait_A	3143.31343159983	344.9094462800716	2293.31996764458	3947.460006776056
10	route_counts_A	61.82	1.8552682291302791	58.0	67.0
11	route_avg_wait_B	1712.0223733004761	214.94645757913378	1311.9392794895757	2175.4112634640783
12	route_max_wait_B	3402.348832443965	355.5375588426902	2591.746359174109	4054.4655481342425
13	route_counts_B	43.65	2.12429429755537	40.0	50.0
14	route_avg_wait_C	7.917609908359455	2.0196640046223444	2.8875212335918796	13.315483801309735
15	route_max_wait_C	37.72512996082841	4.955774550265732	24.070758365881375	48.49841029039658
16	route_counts_C	31.85	6.156830812728709	17.0	47.0
17	queue_max_lengths_hot_service	98.11	17.375179673513273	61.0	141.0
18	queue_time_averages_hot_service	49.259045813724626	10.30774647675756	26.984310009512434	79.08238081785672
19	queue_average_waits_hot_service	1630.0470420537674	204.3782886305903	1107.2904528161475	2066.3623002208446
20	queue_max_waits_hot_service	3226.005055485454	357.9446429694323	2361.221112999711	4066.1319289176513
21	queue_max_lengths_cold_service	84.56	15.380244445128925	49.0	129.0
22	queue_time_averages_cold_service	42.365827297378466	8.403268185915039	26.43076781034645	67.5976597753406
23	queue_average_waits_cold_service	1754.7909727108379	217.7698639638251	1338.1537223779767	2203.341154435507
24	queue_max_waits_cold_service	3491.4854373798503	346.94506237167116	2699.835927649551	4160.159745760071
25	queue_max_lengths_cashier_1	1.06	0.23868325657594...	1.0	2.0
26	queue_time_averages_cashier_1	0.1289527170419822	0.01767596121460...	0.08974146284984...	0.17066597772269...
27	queue_average_waits_cashier_1	5.354768242582228	0.7007106352370363	3.6881760464844495	6.923556247315718
28	queue_max_waits_cashier_1	41.71646785718567	3.472479988292005	33.544505560239486	48.67048199669625
29	queue_max_lengths_cashier_2	1.0	0.0	1.0	1.0
30	queue_time_averages_cashier_2	0.00095234992864...	0.00162856866120...	0.0	0.00899807538981...
31	queue_average_waits_cashier_2	0.5338327741199003	0.9067650445147962	0.0	4.927306355889527
32	queue_max_waits_cashier_2	4.493778440206606	7.887878161119496	0.0	48.49841029039658

Рисунок 2 – Вихідні результати моделювання

На рисунку 3 наведено перші 30 зі 100 результатів запусків базової моделі.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	replication_id	max_in_sys...	avg_in_system	max_total_que...	avg_total_queue	num_customers	route_A_co...	route_A_avg_wait	route_A_max_wait	route_B_co...	route_B_avg_wait	route_B_max_wait	route_C_co...	route_C_avg_wait	route_C_max_wait
2	0	198	111.35825187391244	106	108.44383190375048	137	63	1962.06269321413	3819.674107468254	41	1884.2997728192138	3982.7378382380134	33	5.6467866178454935	35.61892692623633
3	1	201	98.85135615563864	100	95.93804809944548	138	62	1449.5585488750771	2787.6369822665615	44	1622.547590219377	3373.2498887039465	32	7.26225957107378	40.554778202964826
4	2	229	105.00140581776465	124	102.08130424473917	140	61	1594.3934542553523	3251.852286713459 8	50	1704.65148533117 2	3225.983626794965	29	7.0417828105347 93	31.285153220322172
5	3	184	95.88476506435674	95	92.95624328734162	142	62	1551.270660089574	3225.04694501261	44	2113.2487570575413	3777.9087092780155	36	7.6563745786186 23	34.77060742639361
6	4	216	122.23048639843208	124	119.33051261057624	138	59	1907.121098782817	3947.460006776056	43	1858.564387953849	3887.2056658064257	36	7.9201475016281 52	40.84862255829967
7	5	210	102.38595704399835	106	99.40647465578736	144	64	1514.2690830700649	3111.201085685341	43	1833.0220157607184	3693.0354922662036	37	11.346591512699 71	38.7685664253504
8	6	165	84.96054631932171	88	82.07832952145357	134	60	1699.0034825901578	3360.531733335017	47	1581.3678347981338	3237.7365988223373	27	11.601386128935689	33.02012739594238
9	7	209	102.16051366654969	110	99.23292958358658	136	61	1670.6495161816345	2898.3696426686656	44	1925.392298495258 7	3571.1886920397483	31	8.2811446799224 98	36.072348626503754
10	8	140	66.76157144545604	70	63.945582963600685	122	62	1217.2860784589489	2463.155528274182	40	1389.5138413856757	2741.0892070036916	20	5.59099416724267	36.28560575017491
11	9	177	98.65453066053122	96	95.7104108329 32	144	62	1745.7646976487426	3333.0654473599907	49	1860.945836558056	3664.426094100971	33	10.168679674103549	36.184246302210 94
12	10	198	100.04793389973835	106	97.08060711755445	145	64	1607.9060448875596	3169.7582307099615	45	1663.8247867478078	3552.1328506280 58	36	7.2338002805115105	31.603626838868877
13	11	178	89.47687388751623	93	86.53002024795636	142	62	1540.3518686811728	3073.2696033191564	44	1806.965599102 43	3410.653270152199	36	5.8156893771774 88	36.1449814960979
14	12	146	63.993394947517391	73	61.10369302160424	132	62	1069.115430724953	2308.401499478103 7	44	1364.6515225910725	3142.3915141589077	26	7.5475299919626 19	33.387431973414095
15	13	197	95.47966249280627	100	92.62312101535814	127	61	1680.1132272729133	2938.2608015767723	40	1881.0791891188012	3511.406412198896	26	5.5417881811717 63	35.21358460696638
16	14	160	71.07540839146313	83	68.18205501806108	137	61	1349.4359317620742	2724.8580013541296	47	1562.3144214806784	3108.268279136842	29	8.5341544760182 16	39.98876873228983
17	15	165	84.99575866837788	83	82.01821471767543	146	62	1567.1479032632633	3104.9226912787344	45	1634.4109093695374	3444.639432273025	39	6.3909091692381 71	33.72725364593498
18	16	181	85.45468140696 31	105	82.52418221328843	139	61	1508.2828324777056	3271.414004632778	41	1523.585109470554	2939.421842757127	37	5.4960583509608 52	42.3882508570282
19	17	177	89.02775719276744	103	86.1218307376296	134	64	1650.84845404638	3362.692223111194	43	1562.787500726479	3369.8009300966914	27	5.0500895447338 21	29.995749047395293
20	18	183	100.93653741853247	98	98.00421720537666	140	65	1733.9060761555907	3154.028206081375	43	2161.199120611 5317	3969.033807554979	32	10.561682947313411	38.03983613203172
21	19	172	77.12407334872248	104	74.24075819389326	133	59	1476.588711698677	2960.116751537939	44	1415.7935627087345	2817.9123451618457	30	9.025754911594447	32.81287917958616
22	20	190	101.6394780542 74	97	98.75307340376067	137	58	1657.7551809447446	3226.088593280846	40	2075.8986887798724	3819.1973977648604	39	5.6323978668751 63	35.81146556743442 5
23	21	189	88.44649404382191	103	85.43436561855006	149	63	1797.4807676958117	3288.584802580259 7	43	1451.4256033081797	2888.3823824641745	43	8.6205960630728 69	42.71108339117154
24	22	151	69.83834587021582	75	66.91136281844864	132	64	1198.388525741195	2555.699011320852	43	1566.9536598086802	3140.7267657589	25	10.574031675288229	48.49841029039658
25	23	161	79.9085950405768	81	76.94285657867 06	145	63	1406.0339185776793	2293.31996764458	44	1864.4122522700868	3665.4549255391303	38	9.6065442437390 92	41.560539314785274
26	24	200	99.76233869713879	114	96.88047642650764	130	63	1896.6754045692996	3682.099513216416	43	1953.9270773965939	3674.378965683072	24	9.5841241275800 99	43.860504667296425
27	25	152	82.67333406587704	89	79.77412842068945	132	62	1633.136980471673 4	3231.552996198522	43	1722.541371682106	3409.593085995135	27	8.7792660026859 72	38.754362462971585
28	26	158	86.74940354385625	89	83.84193425961809	129	61	1667.1365701456082	3398.7649963915374	43	1775.8804241022447	3278.1575074690436	25	9.0484965278095	40.50336541640581
29	27	214	108.82182167446545	121	105.85426571021226	152	60	1620.1910279572382	3287.0419849284544	45	1600.2173405597537	3130.9929686744968	47	7.5243775232600 1	41.23077669279246
30	28	167	81.16518035460277	86	78.20649883880886	150	63	1286.5764949551321	2868.3789872455654	47	1559.5173635690633	3172.6407555356527	40	8.2447004486013 04	36.55406290834128
31	29	168	95.9512155898232	91	92.99925819046841	142	62	1661.5982303759592	3198.3490609160 93	45	1838.8024160059617	3595.8950609573 35	35	7.4023728023341 05	42.942543620502875
32	30	186	89.29476012856 74	103	86.40076898948 34	137	60	1689.9291524301182	3103.723490457127	46	1499.3766395429272	2995.4608905480145	31	4.8845027999040 52	34.07077104304699

Рисунок 3 – Приклад результатів запуску ітерації базової моделі

На рисунку 4 зображено конфігурації запуску двох варіантів покращеної моделі. Змінювались кількості каналів обслуговування на пунктах гарячих та холодних страв.

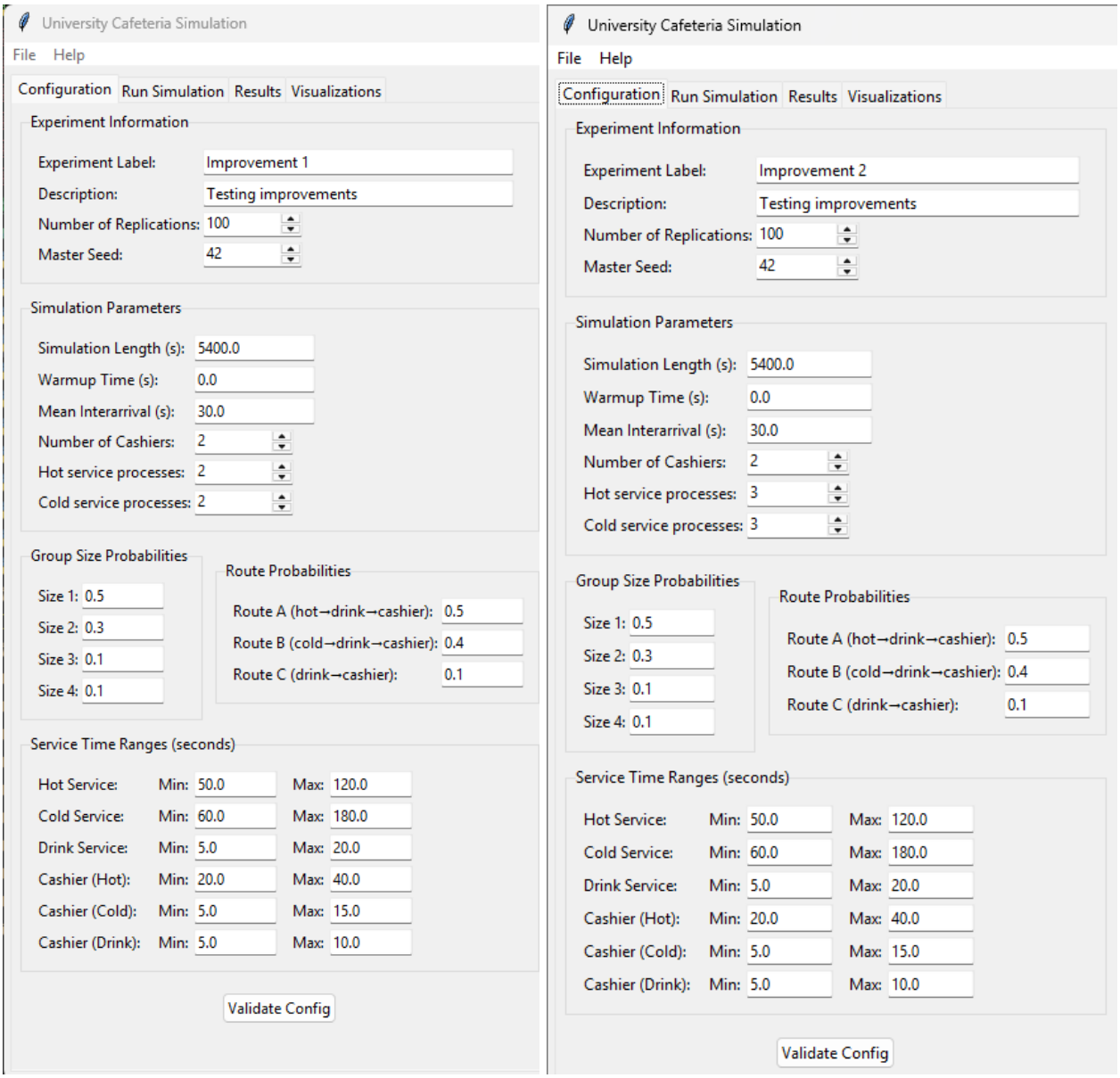


Рисунок 4 – Конфігурації запуску алгоритму симуляції покращених моделей

На рисунку 5 зображено усереднені результати 100 симуляції першої покращеної моделі.

University Cafeteria Simulation

File Help

Configuration Run Simulation Results Visualizations

Summary Statistics

Metric	Mean	Std Dev	Min	Max
max_in_system	88.32	23.99	27.00	151.00
average_in_system	46.70	13.50	17.62	78.23
max_total_queue_length	48.56	13.22	13.00	79.00
time_average_total_queue_length	41.09	13.45	11.99	72.50
global_avg_wait	626.30	175.59	249.01	983.61
global_max_wait	1754.78	392.07	547.93	2732.72
route_avg_wait_A	636.95	272.20	175.78	1306.57
route_max_wait_A	1295.25	477.57	385.37	2480.50
route_counts_A	120.69	4.32	107.00	129.00
route_avg_wait_B	835.49	227.59	200.47	1386.42
route_max_wait_B	1690.05	420.14	529.58	2732.72
route_counts_B	86.26	3.56	73.00	94.00
route_avg_wait_C	11.66	2.13	6.62	17.99
route_max_wait_C	42.18	6.46	25.00	66.85
route_counts_C	31.70	5.93	19.00	52.00
queue_max_lengths_hot_service	41.20	16.53	12.00	79.00
queue_time_averages_hot_service	19.93	9.98	4.34	43.41
queue_average_waits_hot_service	643.96	277.47	169.82	1340.19
queue_max_waits_hot_service	1313.66	489.26	366.57	2583.77
queue_max_lengths_cold_service	42.43	12.28	9.00	77.00
queue_time_averages_cold_service	20.70	6.68	3.80	39.75
queue_average_waits_cold_service	849.17	233.18	197.09	1419.85
queue_max_waits_cold_service	1718.19	431.01	529.82	2732.72
queue_max_lengths_cashier_1	1.93	0.33	1.00	3.00
queue_time_averages_cashier_1	0.41	0.03	0.34	0.50
queue_average_waits_cashier_1	13.06	0.99	11.01	16.23
queue_max_waits_cashier_1	46.83	5.52	40.34	79.39
queue_max_lengths_cashier_2	1.25	0.44	1.00	2.00
queue_time_averages_cashier_2	0.05	0.01	0.02	0.09
queue_average_waits_cashier_2	4.04	0.91	1.82	6.00
queue_max_waits_cashier_2	38.96	5.19	21.85	50.24
server_utilization_hot_service	0.96	0.03	0.85	1.00
server_utilization_cold_service	0.96	0.02	0.89	0.99
server_utilization_drink_station	0.00	0.00	0.00	0.00
server_utilization_cashier_1	0.84	0.02	0.78	0.90
server_utilization_cashier_2	0.32	0.03	0.24	0.40

Рисунок 5 – Результати симуляції з покращеною системою

На рисунку 6 зображено конфігурацію 2 експерименту з покращення системи кафе.

University Cafeteria Simulation

File Help

Configuration Run Simulation Results Visualizations

Summary Statistics

Metric	Mean	Std Dev	Min	Max
max_in_system	35.95	9.49	21.00	71.00
average_in_system	17.82	6.15	9.88	47.91
max_total_queue_length	17.64	6.59	7.00	42.00
time_average_total_queue_length	10.39	5.77	3.13	39.35
global_avg_wait	162.07	77.98	52.46	519.92
global_max_wait	602.28	210.53	247.51	1195.79
route_avg_wait_A	129.33	92.25	33.44	615.76
route_max_wait_A	383.65	160.33	140.52	1036.66
route_counts_A	155.27	14.01	127.00	185.00
route_avg_wait_B	240.21	135.39	46.07	560.33
route_max_wait_B	567.92	224.51	188.64	1195.79
route_counts_B	119.29	7.78	99.00	137.00
route_avg_wait_C	19.95	4.84	9.91	40.19
route_max_wait_C	64.06	15.78	37.64	116.50
route_counts_C	32.55	6.92	19.00	50.00
queue_max_lengths_hot_service	13.53	5.95	5.00	37.00
queue_time_averages_hot_service	3.68	3.44	0.58	23.26
queue_average_waits_hot_service	114.30	90.84	22.82	595.76
queue_max_waits_hot_service	366.07	161.51	130.81	1013.36
queue_max_lengths_cold_service	15.27	6.53	5.00	42.00
queue_time_averages_cold_service	5.72	3.73	0.76	15.32
queue_average_waits_cold_service	225.24	135.12	36.04	536.88
queue_max_waits_cold_service	554.27	222.39	178.42	1165.60
queue_max_lengths_cashier_1	2.89	0.57	2.00	5.00
queue_time_averages_cashier_1	0.68	0.12	0.49	1.34
queue_average_waits_cashier_1	20.31	3.15	15.03	37.33
queue_max_waits_cashier_1	77.67	12.77	53.51	118.94
queue_max_lengths_cashier_2	2.38	0.56	2.00	4.00
queue_time_averages_cashier_2	0.30	0.10	0.17	0.92
queue_average_waits_cashier_2	12.50	2.92	9.00	29.55
queue_max_waits_cashier_2	63.22	14.27	40.99	109.59
server_utilization_hot_service	0.83	0.07	0.66	0.99
server_utilization_cold_service	0.89	0.06	0.71	0.98
server_utilization_drink_station	0.00	0.00	0.00	0.00
server_utilization_cashier_1	0.89	0.03	0.80	0.95
server_utilization_cashier_2	0.62	0.08	0.45	0.83

Рисунок 6 – Результати симуляції з вдруге покращеною системою

Додаток Б. Лістинг програмного коду

У цьому додатку наведено основні фрагменти програмного коду. Повний код програми додатково розміщений за посиланням у GitHub репозиторії: https://github.com/Maximax67/Cafe-Simulation-Coursework

Програмна реалізація просування часу (метод run класу SimulationEngine):

```
def run(self, until: Optional[float]=None, max_events: Optional[int]=None):
    self._running = True
    self._paused = False
    events_processed = 0

    while self._running and not self.event_queue.is_empty():
        if self._paused:
            break

        event = self.event_queue.get_next()
        if event is None:
            break

        if until is not None and event.time > until:
            self._statistic_handler(until)
            break

        if event.time > self.current_time:
            self._statistic_handler(event.time)

        self.current_time = event.time

        handler = self._event_handlers.get(event.event_type)
        if handler:
            handler(event)

        events_processed += 1

        if max_events is not None and events_processed >= max_events:
            break

    self._running = False
```

Програмна реалізація QueueStats, який реалізовує підрахунок значень:

```
class QueueStats:
    queue_name: str
    total_arrivals: int = 0
    total_departures: int = 0
    max_length: int = 0
    waiting_times: List[float] = field(default_factory=list)
    area_under_curve: float = 0.0
    last_change_time: float = 0.0
    last_length: int = 0
    def update_time_average(self, current_time: float, new_length: int):
        time_delta = current_time - self.last_change_time
        self.area_under_curve += self.last_length * time_delta
        self.last_change_time = current_time
        self.last_length = new_length
        if new_length > self.max_length:
            self.max_length = new_length

    def get_time_average_length(self, simulation_end_time: float) -> float:
        time_delta = simulation_end_time - self.last_change_time
        total_area = self.area_under_curve + self.last_length * time_delta
        if simulation_end_time > 0:
            return total_area / simulation_end_time
        return 0.0

    def get_average_waiting_time(self) -> float:
        if len(self.waiting_times) > 0:
            return sum(self.waiting_times) / len(self.waiting_times)
        return 0.0

    def get_max_waiting_time(self) -> float:
        if len(self.waiting_times) > 0:
            return max(self.waiting_times)
        return 0.0
```

Програмна реалізація класу черги:

```
class Queue:
    def __init__(self, name: str):
        self.name = name
        self._queue: deque[Customer] = deque()
        self.stats = QueueStats(queue_name=name)
        self._arrival_times: dict[int, float] = {}  # customer_id -> arrival_time

    def enqueue(self, customer: Customer, current_time: float):
        self._queue.append(customer)
        self._arrival_times[customer.customer_id] = current_time
        self.stats.total_arrivals += 1
        self.stats.update_time_average(current_time, len(self._queue))

    def dequeue(self, current_time: float) -> Optional[Customer]:
        if not self._queue:
            return None
        customer = self._queue.popleft()
        self.stats.total_departures += 1
        arrival_time = self._arrival_times.pop(customer.customer_id,current_time)
        waiting_time = current_time - arrival_time
        self.stats.waiting_times.append(waiting_time)
        self.stats.update_time_average(current_time, len(self._queue))

        return customer

    def peek(self) -> Optional[Customer]:
        if self._queue:
            return self._queue[0]
        return None

    def is_empty(self) -> bool:
        return len(self._queue) == 0

    def length(self) -> int:
        return len(self._queue)

    def get_waiting_time(self, customer_id: int, current_time: float) -> float:
        if customer_id in self._arrival_times:
            return current_time - self._arrival_times[customer_id]
        return 0.0
```

Програмна реалізація базового класу сервера:

```
class Server(ABC):
    def __init__(self, name:str, rng:np.random.Generator, processes:int=1):
        self.name = name
        self.rng = rng
        self.queue = Queue(name)
        self.processes = processes
        self.busy_processes = 0
        self.current_customers: list[Optional[Customer]] = [None] * processes
        self.last_service_start_times: list[float] = [0.0] * processes
        self.total_busy_time = 0.0
        self.service_times: list[float] = []

    @abstractmethod
    def generate_service_time(self, customer: Customer) -> float:
        pass

    def can_serve_immediately(self) -> bool:
        return self.busy_processes < self.processes

    def start_service(self,customer,current_time:float)->Tuple[float,int]:
        if not self.can_serve_immediately():
            raise RuntimeError("No available process.")
        process_index = self.current_customers.index(None)
        self.current_customers[process_index] = customer
        self.last_service_start_times[process_index] = current_time
        self.busy_processes += 1
        service_time = self.generate_service_time(customer)
        self.service_times.append(service_time)
        return service_time, process_index

    def end_service(self, process_index: int, time: float) -> Customer:
        customer = self.current_customers[process_index]
        if customer is None:
            raise RuntimeError(f"Process is not serving any customer.")

        service_duration = time - self.last_service_start_times[process_index]
        self.total_busy_time += service_duration
        self.current_customers[process_index] = None
        self.last_service_start_times[process_index] = 0.0
        self.busy_processes -= 1
        return customer
```

```
    def get_utilization(self, simulation_time: float) -> float:
        if simulation_time > 0 and self.processes > 0:
            return self.total_busy_time / (simulation_time * self.processes)
        return 0.0

    def get_average_service_time(self) -> float:
        if len(self.service_times) > 0:
            return sum(self.service_times) / len(self.service_times)
        return 0.0
```

Програмна реалізація класу CashierBank:

```
class CashierBank:
    def __init__(self, num_cashiers: int, rng: np.random.Generator, **kwargs):
        self.cashiers=[Cashier(i,rng,**kwargs) for i in range(1,num_cashiers+1)]
        self.num_cashiers = num_cashiers

    def select_shortest_queue(self) -> Cashier:
        min_length = float("inf")
        selected_cashier = self.cashiers[0]

        for cashier in self.cashiers:
            queue_length = cashier.queue.length()
            if queue_length < min_length:
                min_length = queue_length
                selected_cashier = cashier

        return selected_cashier

    def get_all_cashiers(self) -> list[Cashier]:
        return self.cashiers
```

Enum можливих маршуртів слідування:

```
class RouteType(Enum):
    ROUTE_A = "A"  # hot → drink → cashier
    ROUTE_B = "B"  # cold → drink → cashier
    ROUTE_C = "C"  # drink → cashier
```

Клас маршруту слідування клієнта:

```
@dataclass
class Route:
    route_type: RouteType
    visit_hot: bool = False
    visit_cold: bool = False
    visit_drink: bool = True
    visit_cashier: bool = True

    @staticmethod
    def create_route_a() -> "Route":
        return Route(
            RouteType.ROUTE_A,
            visit_hot=True,
            visit_cold=False,
            visit_drink=True,
            visit_cashier=True,
        )

    @staticmethod
    def create_route_b() -> "Route":
        return Route(
            RouteType.ROUTE_B,
            visit_hot=False,
            visit_cold=True,
            visit_drink=True,
            visit_cashier=True,
        )

    @staticmethod
    def create_route_c() -> "Route":
        return Route(
            RouteType.ROUTE_C,
            visit_hot=False,
            visit_cold=False,
            visit_drink=True,
            visit_cashier=True,
        )
```

Програмна реалізація класу ExperimentRunner:

```
class ExperimentRunner:
    def __init__(self, experiment_config: ExperimentConfig):
        self.config = experiment_config
        self.replications: List[ReplicationStats] = []
        self.aggregated_stats: Optional[AggregatedStats] = None
        self._cancel_requested = False

    def run(
        self,
        parallel: bool = False,
        max_workers: Optional[int] = None,
        progress_callback: Optional[Callable[[int, int], None]] = None,
    ) -> AggregatedStats:
        self._cancel_requested = False
        self.replications = []

        if parallel:
            results = self._run_parallel(max_workers, progress_callback)
        else:
            results = self._run_sequential(progress_callback)

        self.replications = results
        self.aggregated_stats = AggregatedStats(results)

        return self.aggregated_stats

    def _run_sequential(
        self, progress_callback: Optional[Callable[[int, int], None]] = None
    ) -> List[ReplicationStats]:
        results = []
        total = self.config.num_replications
        for rep_id in range(total):
            if self._cancel_requested:
                break
            stats = run_single_replication(self.config.sim_config, rep_id)
            results.append(stats)
            if progress_callback:
                progress_callback(rep_id + 1, total)

        return results
```

```
def _run_parallel(
    self,
    max_workers: Optional[int] = None,
    progress_callback: Optional[Callable[[int, int], None]] = None,
) -> List[ReplicationStats]:
    if max_workers is None:
        max_workers = mp.cpu_count()
    results = []
    total = self.config.num_replications
    completed = 0
    with ProcessPoolExecutor(max_workers=max_workers) as executor:
        futures = {
            executor.submit(
                run_single_replication, self.config.sim_config, rep_id
            ): rep_id
            for rep_id in range(total)
        }

        for future in as_completed(futures):
            if self._cancel_requested:
                for f in futures:
                    f.cancel()
                break
            rep_id = futures[future]
            try:
                stats = future.result()
                results.append(stats)
                completed += 1
                if progress_callback:
                    progress_callback(completed, total)
            except Exception as exc:
                print(f"Replication {rep_id} generated an exception: {exc}")
    results.sort(key=lambda x: x.replication_id)
    return results

def cancel(self):
    self._cancel_requested = True

def get_results(self) -> Optional[AggregatedStats]:
    return self.aggregated_stats
```

Метод, який відповідає за прибуття групи клієнтів:

```
def _handle_group_arrival(self, event):
        group_size = self.rng.choice([1, 2, 3, 4], p=self.config.group_size_probs)
        group = Group(
           group_id=self.next_group_id, arrival_time=current_time, size=group_size
        )
        self.next_group_id += 1
        for _ in range(group_size):
            route = self._select_route()

            customer = Customer(
                customer_id=self.next_customer_id,
                group_id=group.group_id,
                arrival_time=current_time,
                route=route,
            )
            self.next_customer_id += 1
            group.customers.append(customer)
            self.customers_in_system[customer.customer_id] = customer
            self.stats_collector.update_system_count(current_time, +1)
            self._route_customer_to_first_service(customer, current_time)
        next_arrival = current_time + self.rng.exponential(
            self.config.mean_interarrival
        )
        if next_arrival < self.config.simulation_length:
            self.engine.schedule_event(next_arrival, EventType.ARRIVAL_GROUP)
        if (
            current_time >= self.config.warmup_time
            and not self.stats_collector._collecting
        ):
            self.stats_collector.start_collecting()
```

Метод вибору випадкового маршруту слідування:

```
def _select_route(self) -> Route:
        route_choice = self.rng.choice([0, 1, 2], p=self.config.route_probs)
        if route_choice == 0:
            return Route.create_route_a()
        if route_choice == 1:
            return Route.create_route_b()
        return Route.create_route_c()
```

Метод початкового направлення клієнта по маршруту:

```
def _route_customer_to_first_service(self, customer: Customer, current_time: float):
        if customer.route.visit_hot:
              self.engine.schedule_event(current_time, EventType.ARRIVAL_TO_HOT, customer)
        elif customer.route.visit_cold:
            self.engine.schedule_event(
                current_time, EventType.ARRIVAL_TO_COLD, customer
            )
        else:
            self.engine.schedule_event(
                current_time, EventType.ARRIVAL_TO_DRINK, customer
            )
```

Класи конфігурацій запуску симуляції:

```
@dataclass
class SimulationConfig:
    simulation_length: float = 5400.0  # 90 minutes
    warmup_time: float = 0.0  # No warmup by default
    mean_interarrival: float = 30.0
    group_size_probs: List[float] = field(default_factory=lambda: [0.5, 0.3, 0.1, 0.1])
    route_probs: List[float] = field(default_factory=lambda: [0.5, 0.4, 0.1])
    hot_service_processes: int = 1
    cold_service_processes: int = 1
    hot_service_range: Tuple[float, float] = (50.0, 120.0)
    cold_service_range: Tuple[float, float] = (60.0, 180.0)
    drink_service_range: Tuple[float, float] = (5.0, 20.0)
    cashier_base_overhead: float = 0.0
    cashier_hot_range: Tuple[float, float] = (20.0, 40.0)
    cashier_cold_range: Tuple[float, float] = (5.0, 15.0)
    cashier_drink_only_range: Tuple[float, float] = (5.0, 10.0)
    num_cashiers: int = 2
    master_seed: int = 42

    def __post_init__(self):
       assert len(self.group_size_probs)==4,"Must have 4 group size probabilities"
        assert (
            abs(sum(self.group_size_probs) - 1.0) < 1e-6
        ), "Group size probs must sum to 1"
        assert len(self.route_probs) == 3, "Must have 3 route probabilities"
```

```
        assert abs(sum(self.route_probs) - 1.0)<1e-6,"Route probs must sum to 1"
        assert self.num_cashiers >= 1, "Must have at least 1 cashier"
        assert self.simulation_length > 0, "Simulation length must be positive"
        assert self.warmup_time >= 0, "Warmup time cannot be negative"
        assert (
            self.hot_service_processes > 0
        ), "Hot service processes should be positive"
        assert (
            self.cold_service_processes > 0
        ), "Cold service processes should be positive"

@dataclass
class ExperimentConfig:
    label: str
    sim_config: SimulationConfig
    num_replications: int = 100
    description: str = ""

    def __post_init__(self):
        assert self.num_replications >= 1, "Must have at least 1 replication"
```